Explore or Confirm?
LLM Agents Misallocate Measurements Under Noise

Abdul Karim*
Xstendyl
abdul.karim@xstendyl.com

Matt Keon
Xstendyl
matt.keon@xstendyl.com

Abstract

As model reasoning gets cheaper, the physical experiment becomes the slow and costly step in discovery, and each one returns a noisy reading: a high value can mean a good candidate or a lucky draw. With a fixed budget, an AI agent must keep choosing between exploring a new candidate and confirming a promising one by re-testing it. We measure that choice in Wellocate, a simulated assay campaign with known true values, built on a generated landscape and on two real protein datasets, GB1 and PhoQ, scrambled so no model can answer from memory. We test five language models as minimal agents, with no tools and no memory beyond the history in each prompt. They misallocate in both directions. Three over-confirm, spending most of the budget re-testing. On the generated landscape one under-confirms, trusting single readings, and there every model ends with a worse final pick than a basic scripted strategy. Each failure has its own cause. The over-confirmers run short of new candidates worth trying: a list of untried candidates cuts their re-testing and, on real protein data, makes two of them pick much better. A running summary of their readings or a description of the landscape changes it by at most five percentage points. For the under-confirmer, re-testing its top candidates in a final round, replayed on its own campaigns, recovers only about a quarter of what it loses by picking the wrong candidate, which suggests confirmation has to run through the whole campaign. We release Wellocate as a benchmark and reinforcement-learning environment with frozen splits and scripted baselines, so anyone can measure how an agent splits its budget before trusting it with real experiments, and train agents to split it better. Code, data and the environment are available at `https://github.com/Xstendyl/wellocate`.

1 Introduction

As model reasoning gets cheaper and faster, the physical experiment becomes the bottleneck in discovery. Assays, syntheses and fabrication runs keep their cost and their wall-clock time, and each one returns a noisy reading. So the value of an AI agent in science depends more and more on how it spends a fixed experiment budget. In a protein or drug discovery lab, each round someone, or now some agent, decides which variants go to the assay: a new candidate, or another reading of one that looked good. This is the choice we study in this paper: explore or confirm.

A reading that comes back high may be a good candidate or a lucky draw. Re-testing tells which, and spends a slot that could have tried something new. A campaign is judged like a batting average: what counts is the useful output of the whole budget, and a test that finds nothing costs only its slot. So a re-test earns its slot only if it is worth more than the new candidate it displaces. How

*Corresponding author.

often to repeat a measurement is an old question in experimental design. Sequential design methods choose at each step between a new point and a repeat of an old one, and repeats tend to pay off where the noise is large [Binois et al., 2019]. Batch Bayesian optimization goes a step further and picks both the candidates in each batch and how many times to measure each one [Dai et al., 2023]. Self-driving labs already choose experiments by algorithm [Rapp et al., 2024, Tom et al., 2024], and language-model agents now design experiments [Boiko et al., 2023, Roohani et al., 2025] and proteins [Kim and Romero, 2026]. To our knowledge, how such agents split a noisy budget has not been measured against known true values.

We build Wellocate to measure this choice: a simulated assay campaign in which every candidate's true value is known. It runs on a generated landscape, which we call the grid, and on two real protein landscapes, GB1 and PhoQ [Wu et al., 2016, Podgornaia and Laub, 2015], chosen because protein assays are exactly this kind of slow, costly and noisy step; their amino-acid names are scrambled so no model can answer from memory. We run five language models on the grid, GB1 and PhoQ. Each runs as a minimal agent with no tools and no memory beyond the history of its orders and readings, which it is shown each round, so that what we measure is the model's own allocation.

To say a model re-tests too much or too little, we need a yardstick. We use three simple fixed strategies, the *scripted rules*, that face the same budget and noise as the models. The *guessing rule* tries small changes to its best candidate, guessing which ones are viable (most candidates do not work, and an order for one that does not is rejected), and re-tests that best a fixed number of times each round, set for each landscape. The *model-guided rule* is smarter about what to try: it fits a simple model to its own readings, estimating how much each letter at each position adds to a candidate's value, and orders the new candidates that model predicts will be best. It re-tests one leading candidate each round and spends its last three rounds re-testing its four best. The *informed rule* plays like the guessing rule and is also told which candidates are viable; it still has to measure how good they are. It is never rejected, so it loses no slots to wrong guesses, an advantage the models do not have. On the grid and PhoQ it makes the best final pick. On the grid the guessing rule spends about 24% of its slots on re-tests and the model-guided rule about 38%, with a better final pick. Together these two mark the *scripted band* of re-testing on the grid. Against that band the models fall into three groups (Figure 1). Three over-confirm: Opus 4.7, Gemini 2.5 Flash and DeepSeek V3.1 spend between 75% and 94% of their slots re-testing. One under-confirms: Gemini 3.1 Pro re-tests only 14% and often picks its winner on a single reading. GPT-6.1 Sol, at 44%, sits just above the band. On the grid, every model picks a worse final candidate than the guessing rule, whether it re-tests too much or too little. On GB1 and PhoQ the best scripted rule re-tests at most 1% of its slots, and every model re-tests more than it does, Gemini 3.1 Pro included (Section 3.1, Table 1).

Each failure has its own cause. We give the agents three simple aids: a running summary of their own readings (the ledger), a note on how the landscape is built, and a list of untried candidates (the Menu). The ledger and the landscape note change how much they re-test by at most five percentage points. The Menu cuts re-testing in all three over-confirming models. On GB1, without the Menu, Gemini 2.5 Flash and DeepSeek V3.1 test on average only 3.9 and 2.4 different candidates in a 60-measurement campaign; given the Menu, their final picks close most of the gap to the guessing rule. These models re-test because they run short of new candidates worth trying, so the fix is to supply candidates. Gemini 3.1 Pro trusts single readings. To test the simplest fix, we replay its campaigns and replace its last round of orders with re-tests of its top candidates. This recovers only about a quarter of what it loses by picking the wrong candidate, which suggests confirmation has to run through the whole campaign. This paper contributes:

- A measurement of how five models split a noisy budget (Section 3.1). Their loss divides into

exploration loss, from never measuring better candidates, and selection loss, from picking the wrong one among those measured.

- Evidence on what drives each failure. The over-confirmers run out of new candidates worth trying: the Menu cuts their re-testing, and a summary of their readings or a note on the landscape changes it by at most five percentage points (Section 3.2). The under-confirmer picks winners on one noisy reading, and a final round of re-tests recovers only about a quarter of that loss, suggesting confirmation must run through the whole campaign (Section 3.3). Section 5 turns this into steps a lab can follow.
- Wellocate itself, released as a benchmark and reinforcement-learning environment with a protein track built from real data, frozen splits and scripted baselines, so anyone can measure how an agent splits its budget before trusting it with real experiments, and train agents to split it better (Section 4).

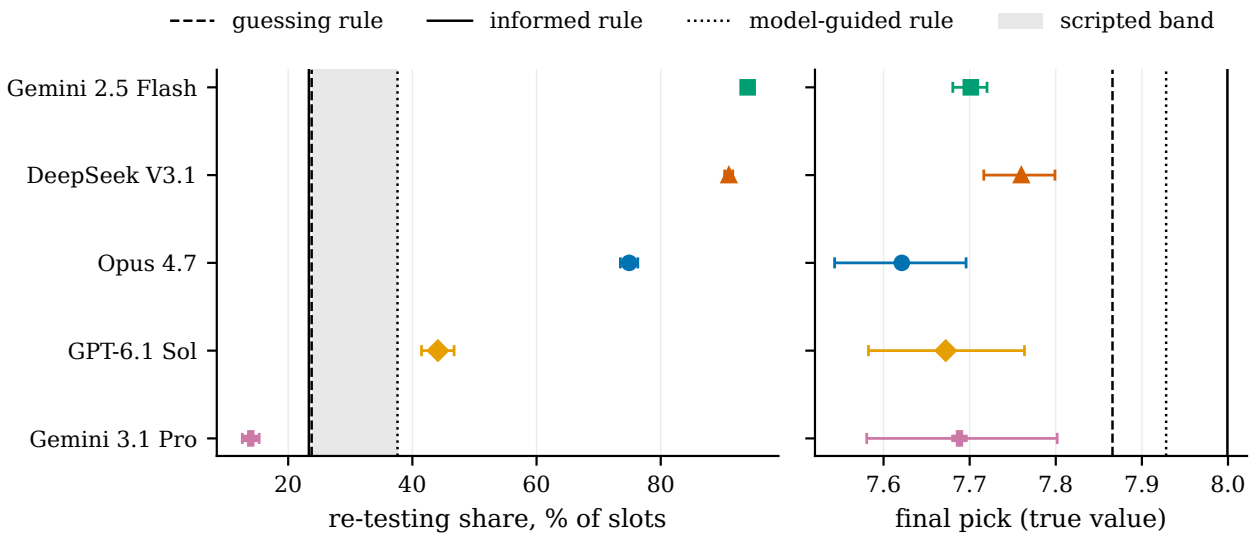


Figure 1: Five models on the grid. Left: the re-testing share, the fraction of used slots spent on a candidate the agent had already ordered, with 95% intervals. Right: the true value of each model's final pick. The lines are the three scripted rules (Section 2), drawn the same way in both panels: the guessing rule (dashed), the model-guided rule (dotted) and the informed rule, which is also told which candidates are viable (solid). The grey band is the scripted band, from the guessing rule's share to the model-guided rule's. The informed rule re-tests about as often as the guessing rule, so their lines overlap on the left. On the grid, Opus 4.7, Gemini 2.5 Flash and DeepSeek V3.1 re-test far above the band, GPT-6.1 Sol just above it and Gemini 3.1 Pro below it, and every model's pick falls short of all three scripted rules. The figure and its band are for the grid only; on GB1 and PhoQ the best scripted rule re-tests at most 1% of its slots (Table 1).

2 Wellocate: a simulated assay campaign

Wellocate turns an assay campaign into an episode with a score (Figure 2). We copy its shape from two real protein experiments, deep mutational scans that set out to measure every variant at four positions of GB1 and of PhoQ [Wu et al., 2016, Podgornaia and Laub, 2015]. A candidate is a string of four letters, one per position, from a 20-letter alphabet, one letter for each amino acid, so the space holds 160,000 candidates. Each candidate has a hidden true value, its quality, which the agent never sees.

The true values come from a landscape, a fixed table that gives every candidate its value. We use three. One is synthetic: a generated NK landscape [Kauffman and Weinberger, 1989] that we call the grid (Figure 3). Each of its four positions gets a score between 0 and 1 from a fixed random table, and the score depends on that position's letter and the next position's letter. A candidate's true value is the average of its four scores. The main study runs on the grid because every true value is known exactly and no model can have seen its optimum. Two are real: the published landscapes of GB1 [Wu et al., 2016] and PhoQ [Podgornaia and Laub, 2015]. We run the same agents on them to check that what we find on the grid also holds on real biology. A model may have memorised these, so their amino-acid names are replaced by the letters A to T through a random map at each position. Each variant keeps its measured value under its new name, and two variants that differ by a single mutation in the real protein still differ by a single letter after the renaming. The scrambled landscape is therefore exactly as hard to search as the real one, and a model cannot answer from memory. Each landscape has its own best candidate (`IOJO` on the grid). We multiply every value in a landscape by one constant so that its best candidate scores 8.762, the top reading in the GB1 data; this puts all three on one scale and changes no ranking. The two kinds of landscape also become the two tracks of the benchmark we release (Section 4): the *grid* track generates fresh NK landscapes, and the *protein* track uses fresh scrambles of GB1 and PhoQ.

Most candidates do not work at all. In a real protein, most random variants do not fold or lose their function, and a search has to find the few that work. We call a candidate *viable* if its true value is in the top 7% of its landscape, and use the same 7% on all three landscapes (Figure 3a). The share is close to the share of GB1 variants that keep at least a tenth of the wild type's fitness (7.2%). The agent is not told which candidates are viable. Viable candidates cluster, so the neighbours of a working candidate are a good place to look. Take any viable candidate and its 76 neighbours, the candidates one letter away (19 other letters at each of 4 positions): on the grid, 25.8% of those neighbours are viable on average, against 7% of all candidates (Figure 3b). The two real landscapes cluster more strongly (45.9% on GB1, 37.6% on PhoQ), so the grid does not make the search easier than real data would.

A campaign has 15 rounds of 4 slots, one slot per assay well, 60 measurements in all. Every campaign starts from the same candidate, the median viable one (`GDIF` on the grid), and the agent is told its value, as a lab starts from a variant it already knows. One shared start gives every model the same search. Each round the agent orders four candidates, and the environment, which holds every true value, handles each order. It rejects an order for a non-viable candidate before the order uses a slot and asks the agent for a replacement; if three replacements in a row are also rejected, the slot is lost. It fails about 15% of orders for viable candidates at random, whatever their value: nothing comes back, and the slot is still spent. Every other order returns a reading: the candidate's true value plus noise with a standard deviation of 7.3% of that value. The 7.3% is the counting error of the GB1 deep mutational scan [Wu et al., 2016]: on average it read each variant's DNA about 557 times before selection and 285 times after, and counting at those depths gives a relative error of about 7.3% (Appendix A). The environment keeps the record of every order, rejection, failure and reading, and shows the agent the full record before each round. The agent may order a candidate it has already measured. At the end it nominates one candidate, scored on its true value. The budget and the failure rate are design choices: the budget is sized for slow assays in which each measurement is costly.

Each piece of the campaign has a lab counterpart. A slot is an assay well. A failed order is a failed assay. A non-viable candidate is a variant that does not fold or express, caught here before it uses a well. The noise is replicate variability: the same variant measured twice gives two different numbers. The nominee is the variant taken forward. Appendix A lists where the environment is

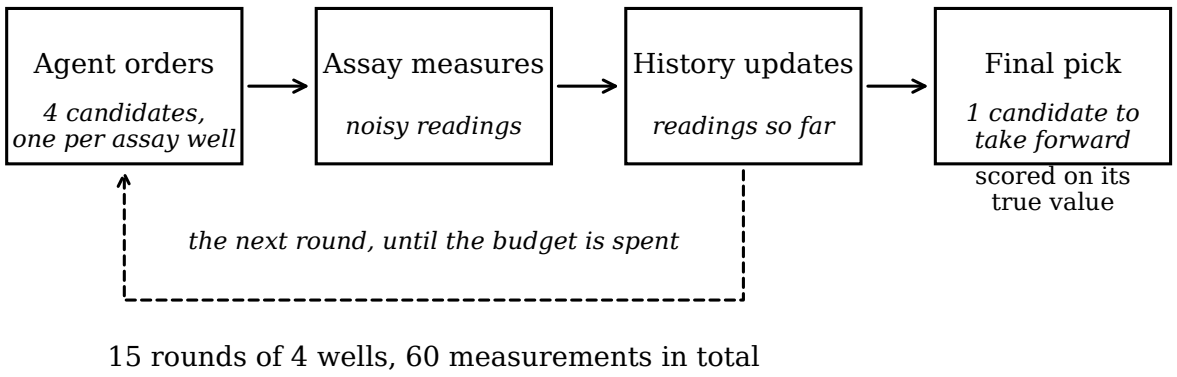


Figure 2: One Wellocate campaign, with the lab counterpart of each part. Each round the agent orders four candidates, one per assay well. The simulated assay returns a noisy reading for each, and the agent sees every reading so far before ordering again. Orders for non-viable candidates are rejected and replaced, and about 15% of orders for viable candidates fail. After 15 rounds the agent nominates the candidate to take forward, scored on its true value.

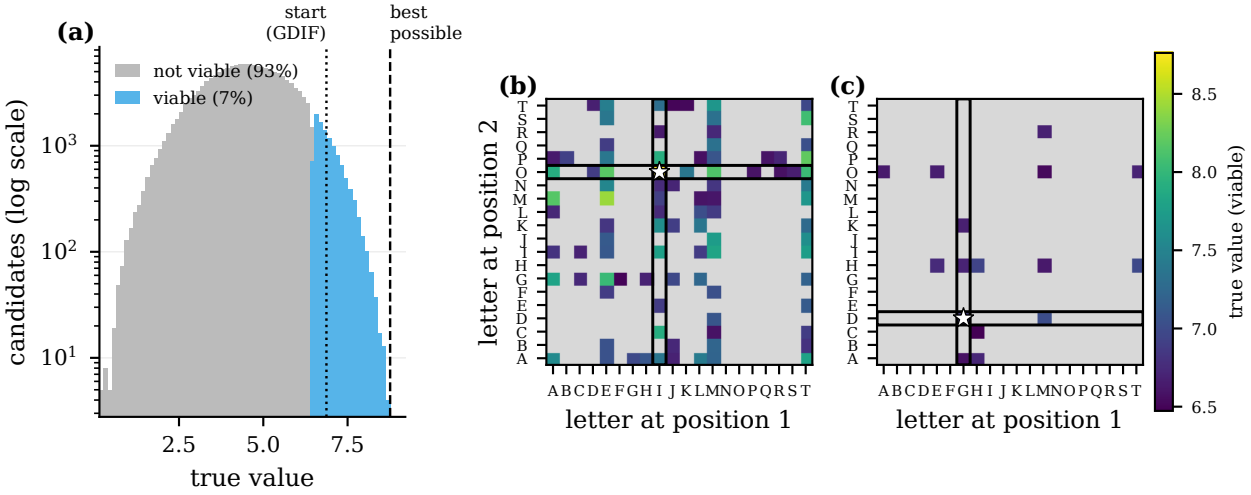


Figure 3: The grid. (a) True value of all 160,000 candidates. A value averages four random position scores, so most candidates sit near the middle and very high or very low values are rare; the vertical axis is on a log scale so that the rare ones show. Only the top 7% (11,199) are viable, shown in blue; an order for any other candidate is rejected. Every campaign starts from `GDIF`, the median viable candidate (6.87), and the best possible is 8.762. (b, c) A candidate has four letters, so each grid shows one slice: letters 3 and 4 are fixed and letters 1 and 2 vary, 400 candidates per slice. Grey cells are not viable. Changing only letter 1 moves along the star's row, and changing only letter 2 moves along its column, so the outlined row and column hold the 38 candidates one letter away that a slice can show. (b) The slice through the best candidate, `IOJO`, with J and O fixed: 23% of the slice is viable, and 20 of the star's 38 outlined neighbours. This is the richest region of the grid. (c) The slice through the start, `GDIF`, with I and F fixed: 16 of 400 cells are viable, and 4 of the star's 38 outlined neighbours. 20 of the start's 24 viable neighbours change letter 3 or 4 and so lie off this slice. The search problem is getting from (c) to (b).

simpler than a lab, and the direction of each simplification.

A finished campaign leaves two things to score: how the agent spent its slots, and how good its final pick was. To score how the slots were spent, we use the *re-testing share*, the fraction of used slots spent on a candidate the agent had already ordered. An agent that orders the same candidate

in all four slots of a round re-tests three of them. The share counts repeats over the whole campaign, whichever candidates they went to; how many readings lie behind the final pick is a separate count, reported in Section 3.1. To score the final pick, we split the final pick's shortfall from the best candidate in the landscape into two parts that add up, both computed from true values, which the agent never sees. The *exploration loss* is how far the best candidate the agent measured falls short of the landscape's best; it is large when the search never reached the good candidates. The *selection loss* is how far the nominated candidate falls short of the best candidate the agent measured; it is large when the agent found a good candidate and then nominated a worse one. Selection loss comes from noise: the agent can rank candidates only by their readings, and a candidate measured once can read well above its true value and look better than one that is truly better. Re-testing averages this noise away and shrinks the selection loss. Each re-test also takes a slot from a new candidate, which can raise the exploration loss.

A re-testing share on its own does not say whether an agent re-tests too much or too little, so we need a yardstick. We use three simple fixed strategies, together called the *scripted rules*: the guessing rule, the informed rule and the model-guided rule. All three work under the same budget, noise and order format as the agents. Each round, the *guessing rule* re-tests its current best a fixed number of times and fills the other slots with untried neighbours of it, candidates one letter away, guessing which ones are viable, as the agents must. The *informed rule* does the same and is also told which candidates are viable, so it is never rejected; it still has to measure how good they are. For these two rules we set the number of re-tests per round after the runs, to the one with the best mean true pick on each landscape, a tuning the agents do not get. The *model-guided rule* learns from its own readings only. It scores each letter at each position by the mean reading of the measured candidates that carry it, and adds these scores to rank untried one- and two-letter changes of its current best. It re-tests one leading candidate each round and spends its last three rounds re-testing its four best. Appendix B gives the settings of all three.

Against this yardstick we run five language models: Opus 4.7, Gemini 2.5 Flash, DeepSeek V3.1, GPT-6.1 Sol and Gemini 3.1 Pro, each as a minimal agent. Every round it receives one prompt holding the task and the full history of its orders and readings, and it replies with its orders. That history, written by the environment, is its only memory of earlier rounds: the model itself carries nothing from one call to the next, and it has no tools or scaffold. We chose this on purpose, so that what we measure is the model's own allocation.

3 Results

3.1 Misallocation in both directions

The five models, run under a registered design (Appendix D), misallocate their measurements in both directions.

The scripted rules set the yardstick (Table 1). On the grid the guessing and model-guided rules re-test 23.78% to 37.61% of their slots, the *scripted band*. The best final pick comes from the informed rule, which is told which candidates are viable: a candidate worth 8.00. On the two proteins, at the noise of the real assay (*assay noise*), the best scripted rule hardly re-tests at all: 0.9% of slots on GB1 and none on PhoQ. With more noise, re-testing pays there too (Section 4).

Three models, Opus 4.7, Gemini 2.5 Flash and DeepSeek V3.1, over-confirm. On the grid they spend 75% to 94% of their slots re-testing, far above the scripted band (Figure 1), and with every reading exact they still re-test half to nearly all of their slots, far more than exact readings need (Appendix H). Gemini 3.1 Pro under-confirms. It re-tests only 14% of its slots, below the band, so

Table 1: Re-testing share and true value of the final pick on each landscape, averaged over each model's campaigns. The three scripted rules are shown at their best setting.

	grid		GB1		PhoQ	
	share (%)	pick	share (%)	pick	share (%)	pick
Opus 4.7	74.9	7.62	63.3	4.78	62.8	1.89
Gemini 2.5 Flash	94.0	7.70	91.9	1.12	63.7	2.13
DeepSeek V3.1	91.0	7.76	96.0	0.84	89.9	0.83
GPT-6.1 Sol	44.1	7.67	19.4	6.36	23.9	2.37
Gemini 3.1 Pro	14.0	7.69	11.1	4.90	14.7	1.72
guessing rule	23.8	7.87	0.9	**6.09**	1.0	2.05
informed rule	23.3	**8.00**	0.0	6.02	0.0	**2.63**
model-guided rule	37.6	7.93	22.6	5.61	25.3	1.89

on the grid its pick is usually a candidate it measured only once. GPT-6.1 Sol sits just above the band. On GB1 and PhoQ, where the best scripted rule re-tests at most 1% of its slots, some models re-test less than on the grid, and every model still re-tests more than that rule, Gemini 3.1 Pro included (Table 1). On the grid every model's final pick falls below the guessing rule's 7.87, and re-testing less did not help Gemini 3.1 Pro or Sol pick better (Figure 1). On GB1 Sol's lead over the guessing rule is uncertain, and on PhoQ, a hard landscape for every strategy, three models are about level with it, and DeepSeek V3.1 and Gemini 3.1 Pro pick worse (Appendices E.3 and F).

The over-confirmers lose through exploration (Figure 4): the best candidate they ever measured falls 0.94 to 1.04 short of the landscape's best, against 0.67 for the guessing rule, and among what they tested they pick almost perfectly. Gemini 3.1 Pro loses through selection. It explores about as well as the guessing rule and loses 0.38 at selection, against 0.23 for the guessing rule, because it often picks a candidate it measured only once. Sol loses about as much as the over-confirmers through exploration and less than the guessing rule at selection.

3.2 Over-confirmation

To understand why the three over-confirming models re-test so much (Figure 1), we tested three explanations on the grid. Each comes with an aid, given to these three models on the same seeds as their control, and if an explanation holds, its aid should lower re-testing. GPT-6.1 Sol and Gemini 3.1 Pro do not over-confirm, so these aids do not apply to them.

1. **Ledger.** *They lose track of what they have measured.* The agent sees its history as one line per order, so a candidate's readings are scattered across rounds. The ledger lists each measured candidate with its number of readings, their mean and its standard error.

2. **Landscape note.** *They do not know where to look next.* The agent is told nothing about how the landscape is built. The landscape note states how a candidate's score is built from its letters, and that a viable candidate's neighbours are far more often viable than candidates in general.

3. **Menu.** *They run out of new candidates worth trying.* The agent must name every new candidate itself, and most of the new names it tries are rejected as non-viable. The Menu lists ten candidates the agent has never ordered, each one letter from one of its three best, ranked by a simple model fitted to its own readings.

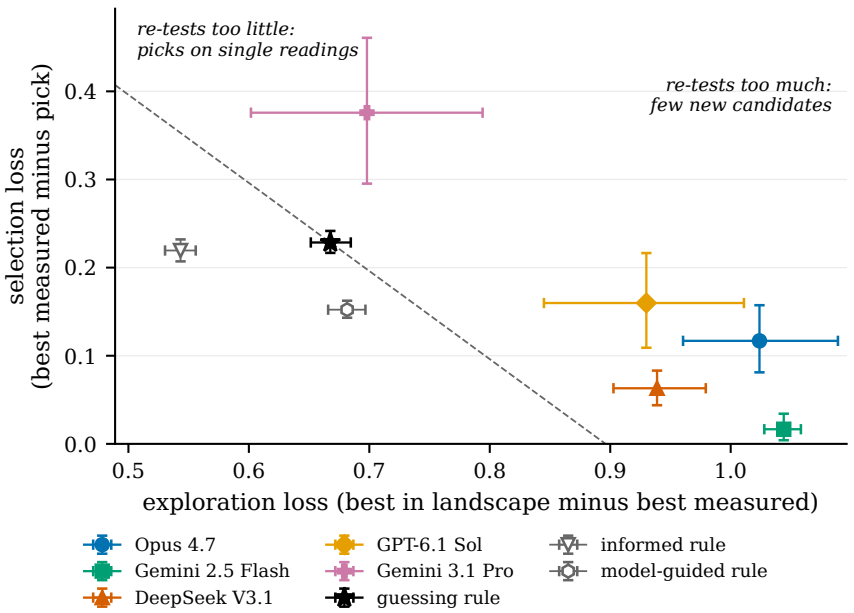


Figure 4: Where each model's loss comes from, with 95% intervals. Exploration loss: how far the best candidate measured falls short of the landscape's best. Selection loss: how far the pick falls short of the best candidate measured. The dashed line joins points with the guessing rule's total loss. The informed and model-guided rules are shown for reference; the informed rule is told which candidates are viable. Opus 4.7, Gemini 2.5 Flash and DeepSeek V3.1 measure too few candidates and pick well among them; Gemini 3.1 Pro measures as widely as the guessing rule and picks poorly.

The ledger and the landscape note changed re-testing by at most five percentage points (Figure 5a). Three of these six changes were rises: the ledger for Opus 4.7 and DeepSeek V3.1, and the landscape note for DeepSeek V3.1. The Menu cut re-testing sharply in all three models, raised the best candidate they measured and worsened their selection (Appendix E.4). The over-confirmers re-test because they run short of candidates worth testing.

We then gave the Menu to all five models on scrambled GB1, built from real protein data, to test whether it carries over and lifts the final pick. It cut re-testing again in the three over-confirmers (Figure 5b). Without it, Gemini 2.5 Flash and DeepSeek V3.1 tested on average only 3.9 and 2.4 different candidates per campaign. With it, their picks rose by more than three units of true value (Figure 5c). Almost all of the gain came from finding better candidates, that is, from a smaller exploration loss.

For Gemini 3.1 Pro and Sol the Menu moved re-testing by at most four points and did not change their picks detectably (Figure 5b and c), so they do not lack candidates. Gemini 3.1 Pro loses in how it picks, on single readings (Section 3.3).

3.3 Under-confirmation

Gemini 3.1 Pro under-confirms: it re-tests too little, so it often picks a candidate it measured only once and loses at selection (Figure 4). If that is the cause, re-testing its top candidates before it picks should win back part of the loss. Under-confirmation shows only on the grid, where re-testing pays, so we ran this test there. We asked one question: had the agent spent its last round re-checking its best candidates, would its final pick have been better? To answer it without calling the model again, we rebuilt each grid campaign of Gemini 3.1 Pro, and of Sol for comparison, from its log, which

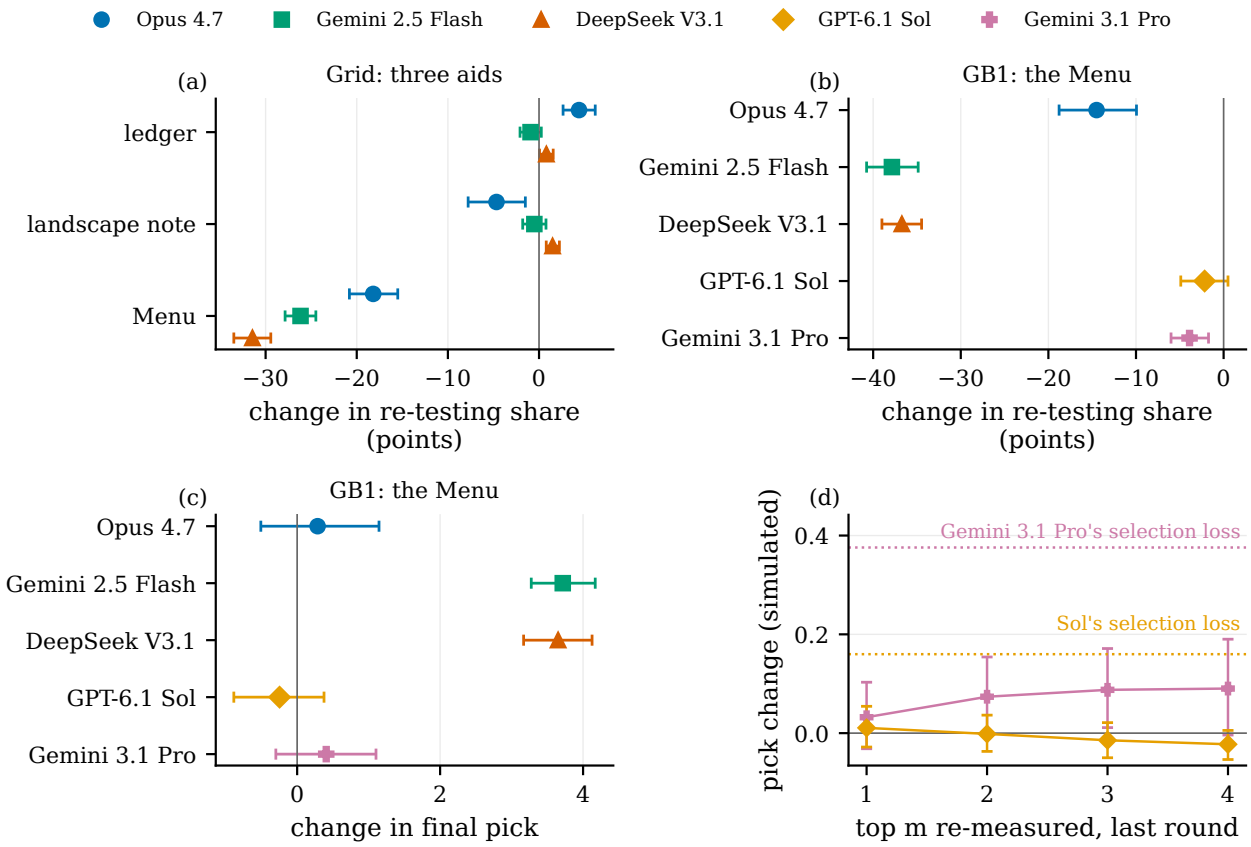


Figure 5: What we tested for each failure, each against the same model's control on the same seeds, with 95% intervals. *Over-confirmation.* (a) The three aids on the grid, given to the three over-confirming models: change in re-testing share. (b, c) The Menu on scrambled GB1, given to all five models: change in re-testing share and in final pick. It cuts re-testing in the over-confirming models and lifts two of their picks by more than three units. It moves the re-testing of Gemini 3.1 Pro and Sol by at most four points. *Under-confirmation.* (d) The grid campaigns of Gemini 3.1 Pro and Sol, replayed with the last round spent re-testing the top m candidates: $m = 1$ re-tests the top one four times, and $m = 4$ re-tests each of the top four once. The y-axis is the change in the true value of the final pick against the agent's own pick. Each dotted line is that model's selection loss, the most better checking could win back. Gemini 3.1 Pro recovers about a quarter of it; Sol gains nothing.

reproduces every reading exactly. We then replaced the final round's four orders with re-tests of the top m candidates by mean reading, under the same noise and failure rate. A fixed rule made the pick, the study's registered selection rule. It ranks candidates by average reading, and pulls the average of a candidate with few readings toward the campaign's overall average, so one lucky reading counts for less.

Gemini 3.1 Pro's pick improves as the re-tests are spread over more candidates, and at $m = 3$ it recovers about 23% of its selection loss (Figure 5d). This fits a true best that often sits just behind a candidate with one lucky reading, so a second reading lets it overtake. The same rule applied to its unchanged readings gains almost nothing (Appendix G), so the gain comes from the extra readings. The gain stays small because Pro ends a campaign with dozens of candidates measured only once, and four slots can re-check only a few. Sol gains nothing, since its top candidates already have several readings each. Its curve dips slightly below zero because the replay gives up the new candidates its real final round would have tried. In an exploratory run, re-testing over the last two

rounds recovered about as much as the last round alone.

4 Wellocate as a benchmark and RL environment

Our study measures how an agent splits a noisy budget between new candidates and re-tests. We release Wellocate so others can measure their own agents the same way, and train agents to split the budget better. It runs our campaigns through a standard reinforcement-learning interface, gives language models the exact prompts ours received, and comes with frozen splits of scrambled GB1 and PhoQ and of generated grid landscapes (Appendix C). The environment, its code, the campaign logs and the analysis behind each number in this paper are at `https://github.com/Xstendyl/wellocate`.

An environment for this problem is only worth learning in if the right split depends on the situation. If one fixed rule were best everywhere, there would be nothing to learn. On each landscape, at five noise levels, we ran the guessing rule at every setting from no re-tests to four a round, and recorded how good its final pick was (Figure 6).

Each curve is one noise level, and its peak, marked with a star, is the best number of re-tests. At the real assay's noise the grid peaks at one re-test a round. GB1 and PhoQ peak at none, and every re-test added makes the pick worse. More noise moves the peak: re-testing begins to pay on PhoQ at about 4 times the assay's noise and on GB1 only at about 8 times. The reason is how close the best candidates sit. The ten best grid candidates lie within 0.32 noise standard deviations of the best, against 2.95 on GB1 and 6.80 on PhoQ, so one reading rarely puts protein candidates in the wrong order.

The best fixed rule changes with the landscape and the noise. A good policy has to read the landscape and the noise and set its split from them, and Wellocate asks a learner to do exactly that. This is also why Section 3 judges each model against the best rule for its own landscape and noise.

Last, we asked whether an agent can learn this task inside Wellocate. We trained a small

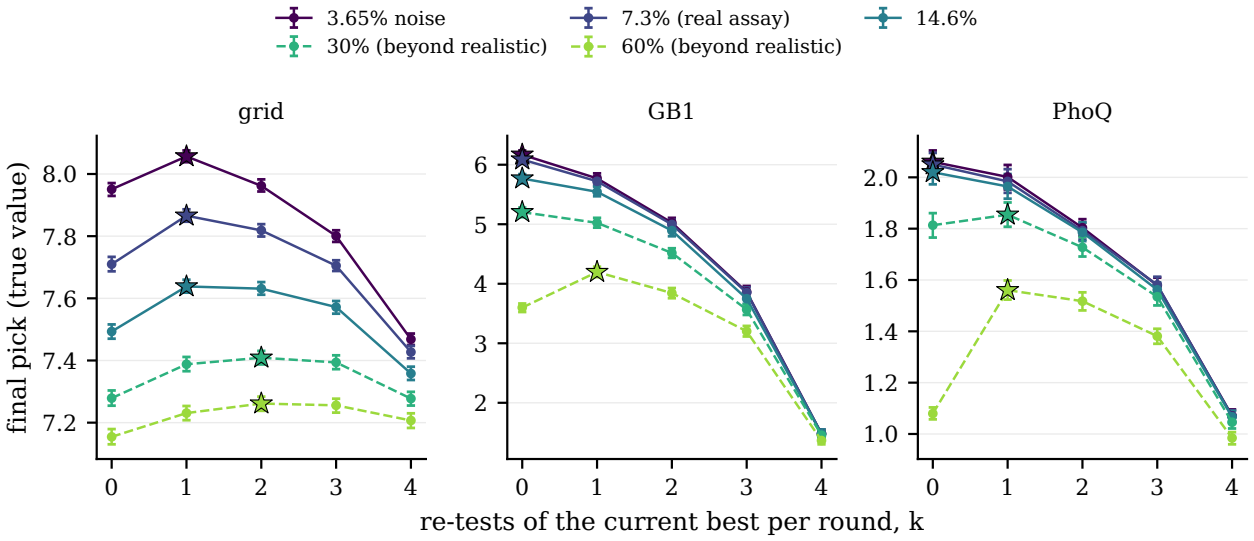


Figure 6: The guessing rule's final pick for each number of re-tests per round, with 95% intervals. Solid curves cover realistic noise; dashed curves go far beyond it, to show where re-testing starts to pay on the proteins. The star marks each curve's best setting; where a curve is flat at the top, the star is a near tie.

policy from scratch with proximal policy optimisation (PPO), a standard reinforcement-learning method, only on generated grid landscapes, so it never saw a protein or a language model's campaign (Appendix L). On grid landscapes it had not seen, its final pick beats a policy that orders untried candidates at random by 0.40 units of true value, and matches the guessing rule. Its lead is not clear on the most rugged generated landscapes. It also works on both proteins, under scrambled and real names, and stays below the best scripted rule there. In a later check, given the same landscapes, seeds, noise and budget as each language model's grid campaigns, it picks better than all five models. Replays show what it learned: it builds new candidates from its best ones and almost never re-tests them. A small learned policy can already beat these models on the grid, and learning when to confirm is still open.

5 Discussion

Our results suggest that current language models do not split a noisy experimental budget well, and that they go wrong in two directions. Three of the five models re-test far more than any scripted rule and run short of new candidates worth trying. Gemini 3.1 Pro re-tests too little and often takes forward a candidate it measured once. Both failures stay hidden in the final answer, which always looks like a reasonable pick. What matters is the useful output of every slot over the whole campaign: an agent that re-tests everything rarely backs a lucky reading, and it also finds little.

The failures sit at the level of the campaign, and more thought about each experiment did not fix them. Gemini 3.1 Pro sometimes reasoned for over 16,000 tokens before one round of orders, and its final pick still had a single reading in 78% of grid campaigns. Opus 4.7 and Gemini 2.5 Flash still re-test most of their slots with reasoning switched on (Appendix I). Prompt wording did not fix them either: the one wording change that passed our registered test shrank to almost nothing on fresh seeds (Appendix J), and a one-sentence lab role lowered re-testing by at most ten points (Appendix I). What helped the over-confirmers was a cheap supply of candidates worth testing (Section 3.2). For the under-confirmer, one final round of checks recovered about a quarter of its loss, which suggests confirmation has to run through the whole campaign (Section 3.3).

For a lab running agents, this suggests three steps.

1. *Measure the split.* Compare the agent's re-testing share with the best scripted share for its landscape and noise, and split its loss into exploration and selection. Wellocate's scorecard reports all three.

2. *Give an over-confirmer candidates.* A cheap candidate generator, such as the Menu, lets it spend the freed slots on new candidates.

3. *Make an under-confirmer re-test along the way.* Re-test a leading candidate every round and the top few at the end, as the scripted model-guided rule does.

The main limits of our study concern the range of landscapes and models we tested, the simple agent we used, and the parts of a real lab that Wellocate leaves out, which Appendix A lists. We compared five models on three landscapes, one generated and two real proteins under scrambled names, so other real landscapes may behave differently. Among the proteins the Menu was tested on GB1 only, and under-confirmation appeared in one model, Gemini 3.1 Pro, on the grid alone. Our agent was deliberately simple: each round it made one fresh call with the full history in its prompt, with no tools and no other memory. An agent with tools, memory or a planning layer may split its budget differently.

6 Related work

Language-model agents already plan and run experiments in chemistry [Boiko et al., 2023]. In biology they design genetic perturbation screens, tested against data from screens already completed [Roohani et al., 2025]. Kim and Romero [2026] test agents on protein design and trace their shortfall to how thoroughly they check candidates. The agents pick suitable tools, score each candidate on only a few measures and stop exploring early. In those tasks chance enters through design tools that sample new candidates. In ours it enters through measurement, so one candidate measured twice gives two different numbers. Wang et al. [2026] add noise to what users say and what tools return in existing agent benchmarks, and measure how well agents hold up.

Krishnamurthy et al. [2024] placed GPT-3.5, GPT-4 and Llama2 in multi-armed bandits, where an agent repeatedly picks one of several options with random payoffs. The prompt held the task and the history of past picks, either raw or summarised. Only GPT-4, reasoning step by step with a summarised history, explored well. Our ledger gives the same kind of summary, and in our study it changed re-testing little (Section 3.2). Their payoffs are coin flips, so the noise level is tied to the payoff and cannot be set on its own.

LLAMBO puts a language model inside Bayesian optimisation, to propose candidates and predict their scores [Liu et al., 2024]. Gupta et al. [2025] tested whether such a model uses the results it gets: shuffling the measured outcomes on genetic perturbation tasks left performance essentially unchanged. On feedback from a real CRISPR screen, Wainrib et al. [2026] found that shuffling removed the gain. In both studies the results come from completed screens, and re-measuring a gene is not offered as a use of the budget.

Experimental design has long studied how to split a budget between new conditions and repeats. Binois et al. [2019] study when repeating a run beats trying a new one in noisy computer simulations. Chen et al. [2000] split a fixed number of simulation runs among a set of options so as to pick the best one. Dai et al. [2023] describe the trade-off our agents face, under large noise whose size varies across conditions. One can test more new conditions with fewer repeats each, or fewer conditions with more repeats. Their method sets the number of repeats for each chosen condition from the data so far. Our scripted rules are simpler: each re-tests its current best a fixed number of times per round. A rule that sets its re-testing from the data would be a stronger yardstick. In bandit terms, a Wellocate campaign is close to best-arm identification on a fixed budget. The task is to find the best option with a set number of noisy tries, here made four at a time, and the score is the true value of the final pick. In Wellocate, candidates that share letters have related values, so one reading says something about other candidates, and the agent may nominate a candidate it never measured.

Wu et al. [2016] measured the fitness of nearly every variant at four sites of the protein GB1 and mapped its fitness peaks. Our GB1 track is built from their data. Wu et al. [2019] used the same landscape to test machine-learning-guided directed evolution, which picks the next variants to measure from a model fitted to earlier readings. Our model-guided rule is a simple version of that idea. Our grid uses the NK model [Kauffman and Weinberger, 1989], a classic generated landscape. Other generated landscapes come from physical models: Lobkovsky et al. [2011] build them from a protein-folding model and ask how predictable evolution is on them.

We find four benchmark environments closest to our work. Wellocate combines four features: landscapes generated by program, measurement noise whose level we set, down to zero, re-testing as a choice that uses budget, and one scored final pick. BoxingGym [Gandhi et al., 2025] is the nearest. Its ten environments are probabilistic models of fixed real-world domains, and the agent's goal is to learn the model. Experiments are scored by expected information gain, how much each is expected to teach about the model. The study does not vary the noise level or test a noise-free

setting, and it does not study repeating an experiment as a choice. Park et al. [2026] also generate their environments by program and aim to isolate specific behaviours of agents. Their observations carry no noise, so a second look returns what the first one showed. Wang and Xu [2026] have agents allocate a fixed budget, under task descriptions that range from abstract to concrete, and their outcomes carry no noise. Huai et al. [2026] study agents that decide from accumulated experimental results, on 30 real machine-learning tasks with no set noise level. Each of these lacks at least one of the four features, and each keeps its noise level fixed.

Acknowledgements

We used AI coding assistants for implementation, analysis and drafting. The authors checked all results and take full responsibility for the content.

References

Mickael Binois, Jiangeng Huang, Robert B. Gramacy, and Mike Ludkovski. Replication or exploration? Sequential design for stochastic simulation experiments. *Technometrics*, 61(1):7–23, 2019. doi: 10.1080/00401706.2018.1469433. arXiv:1710.03206.

Daniil A. Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624(7992):570–578, 2023. doi: 10.1038/s41586-023-06792-0.

Chun-Hung Chen, Jianwu Lin, Enver Yücesan, and Stephen E. Chick. Simulation budget allocation for further enhancing the efficiency of ordinal optimization. *Discrete Event Dynamic Systems*, 10 (3):251–270, 2000. doi: 10.1023/A:1008349927281.

Zhongxiang Dai, Quoc Phong Nguyen, Sebastian Shenghong Tay, Daisuke Urano, Richalynn Leong, Bryan Kian Hsiang Low, and Patrick Jaillet. Batch bayesian optimization for replicable experimental design. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Kanishk Gandhi, Michael Y. Li, Lyle Goodyear, Agam Bhatia, Louise Li, Aditi Bhaskar, Mohammed Zaman, and Noah D. Goodman. BoxingGym: Benchmarking progress in automated experimental design and model discovery. *arXiv preprint arXiv:2501.01540*, 2025.

Rushil Gupta, Jason Hartford, and Bang Liu. LLMs for bayesian optimization in scientific domains: Are we there yet? In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 15482–15510, 2025. doi: 10.18653/v1/2025.findings-emnlp.838.

Tianyu Huai, Tingshuo Fan, Xinchi Chen, Yining Zheng, Yuxin Wang, Shuang Chen, Jie Zhou, and Xuanjing Huang. AgentHPOBench: A benchmark for evaluating LLM agents as sequential hyperparameter optimizers. *arXiv preprint arXiv:2607.29626*, 2026.

Stuart A. Kauffman and Edward D. Weinberger. The NK model of rugged fitness landscapes and its application to maturation of the immune response. *Journal of Theoretical Biology*, 141(2):211–245, 1989. doi: 10.1016/S0022-5193(89)80019-0.

Jeonghyeon Kim and Philip Romero. Benchmarking and behavioral characterization of LLM agents for protein design. *bioRxiv*, 2026. doi: 10.64898/2026.05.06.723381.

Akshay Krishnamurthy, Keegan Harris, Dylan J. Foster, Cyril Zhang, and Aleksandrs Slivkins. Can large language models explore in-context? In *Advances in Neural Information Processing Systems*, volume 37, pages 120124–120158, 2024. doi: 10.52202/079017-3818.

Tennison Liu, Nicolás Astorga, Nabeel Seedat, and Mihaela van der Schaar. Large language models to enhance bayesian optimization. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL `https://openreview.net/forum?id=OOxotBmGol`.

Alexander E. Lobkovsky, Yuri I. Wolf, and Eugene V. Koonin. Predictability of evolutionary trajectories in fitness landscapes. *PLoS Computational Biology*, 7(12):e1002302, 2011. doi: 10.1371/journal.pcbi.1002302.

Jaden Park, Jungtaek Kim, Jongwon Jeong, Robert D. Nowak, Kangwook Lee, and Yong Jae Lee. Exploration and exploitation errors are measurable for language model agents. *arXiv preprint arXiv:2604.13151*, 2026.

Anna I. Podgornaia and Michael T. Laub. Pervasive degeneracy and epistasis in a protein-protein interface. *Science*, 347(6222):673–677, 2015. doi: 10.1126/science.1257360.

Jacob T. Rapp, Bennett J. Bremer, and Philip A. Romero. Self-driving laboratories to autonomously navigate the protein fitness landscape. *Nature Chemical Engineering*, 1(1):97–107, 2024. doi: 10.1038/s44286-023-00002-4.

Yusuf Roohani, Andrew Lee, Qian Huang, Jian Vora, Zachary Steinhart, Kexin Huang, Alexander Marson, Percy Liang, and Jure Leskovec. BioDiscoveryAgent: An AI agent for designing genetic perturbation experiments. In *International Conference on Learning Representations (ICLR)*, 2025. URL `https://openreview.net/forum?id=HAwZGLcye3`.

Gary Tom, Stefan P. Schmid, Sterling G. Baird, Yang Cao, Kourosh Darvish, Han Hao, Stanley Lo, Sergio Pablo-García, Ella M. Rajaonson, Marta Skreta, Naruki Yoshikawa, Samantha Corapi, Gun Deniz Akkoc, Felix Strieth-Kalthoff, Martin Seifrid, and Alán Aspuru-Guzik. Self-driving laboratories for chemistry and materials science. *Chemical Reviews*, 124(16):9633–9732, 2024. doi: 10.1021/acs.chemrev.4c00055.

Gilles Wainrib, Barbara Bodinier, Haitem Dakhli, Josep Monserrat, Almudena Espin Perez, Sabrina Carpentier, Roberta Codato, and John Klein. Can AI scientist agents learn from lab-in-the-loop feedback? evidence from iterative perturbation discovery. *arXiv preprint arXiv:2603.26177*, 2026.

Daniel Wang and Andrew Xu. AllocBench: Measuring online tool allocation capability in LLM agents. *arXiv preprint arXiv:2607.23332*, 2026.

Ruipeng Wang, Yuxin Chen, Yukai Wang, Chang Wu, Junfeng Fang, Xiaodong Cai, Qi Gu, Hui Su, An Zhang, Xiang Wang, Xunliang Cai, and Tat-Seng Chua. AgentNoiseBench: Benchmarking robustness of tool-using LLM agents under noisy condition. *arXiv preprint arXiv:2602.11348*, 2026.

Nicholas C Wu, Lei Dai, C Anders Olson, James O Lloyd-Smith, and Ren Sun. Adaptation in protein fitness landscapes is facilitated by indirect paths. *eLife*, 5:e16965, 2016. doi: 10.7554/eLife.16965.

Zachary Wu, S. B. Jennifer Kan, Russell D. Lewis, Bruce J. Wittmann, and Frances H. Arnold. Machine learning-assisted directed protein evolution with combinatorial libraries. *Proceedings of the National Academy of Sciences*, 116(18):8852–8858, 2019. doi: 10.1073/pnas.1901979116.

Appendices

The appendices fall into four groups. Appendices A to C describe the environment, the scripted rules and how to use Wellocate. Appendices D to G hold the five-model study: what was registered, the runs, PhoQ and the checks behind Section 3. Appendices H to K keep the pre-registered prompt study, its follow-up experiments and the learnability simulation. Appendix L covers the trained policy.

A Environment details

The landscape. A candidate's fitness comes from an NK landscape [Kauffman and Weinberger, 1989]. N is the number of positions, here four, and K is how many other positions each position's contribution depends on. A landscape is rugged when it has local peaks, candidates that no single-letter change can improve, and a larger K makes it more rugged. We set $K = 1$, so each position's contribution depends on the next position along the string, wrapping round at the end. Each position has its own table of contributions, one for every pair of letters at that position and the next, drawn uniformly between 0 and 1 when the landscape is generated. A candidate's fitness is the mean of its four contributions. The whole landscape is then multiplied by one constant so its maximum is 8.762, the highest value measured in the published data. This only sets the units, and beyond that value and its share of working candidates the landscape is a standard NK landscape. At $K = 1$ position interactions are weak, so the search itself is simple, and an agent that falls short does so through how it spends measurements. The agents were run at $K = 1$ only, and the learnability simulation in Appendix K repeats the scripted analysis at $K = 2$ and 3.

Viability. A candidate is viable if its fitness exceeds the 93rd percentile, which leaves 11,199 viable candidates, 7.00% of the space. The threshold was chosen so that about the same share of candidates works as in the published data. In GB1, 7.2% of variants keep at least a tenth of the wild type's fitness. GB1 and PhoQ are cut at the same share, so their never-measured variants, stored as zero, are all non-viable. On PhoQ the authors' own call of functional variants is much narrower, so there the cut also admits variants they would call non-functional. Treating "does not work" as a sharp cutoff on fitness is a simplification.

Ordering and failures. Each slot is filled by an order, a request to measure one candidate once. An order for a non-viable candidate is rejected. The agent is told, and is asked for a replacement in the same round, up to three times. If every replacement is also non-viable, the slot is lost and still counts against the budget. About 15% of orders fail at random, whatever the candidate's fitness, and a failed order returns nothing and still uses its slot. The rate is a design choice. Over the 5,075 agent campaigns of the prompt study (Appendix H), 14.97% of 300,898 measurements attempted on viable candidates failed.

Noise. Each measurement of a viable candidate returns its true fitness plus independent Gaussian error, whose standard deviation grows with the true fitness f:

$$\sigma(f) = \sqrt{0.00351^2 + (0.073f)^2}.$$

Both constants in $\sigma(f)$ are sized from the read counts of Wu et al. [2016], who screened a library spanning all 160,000 variants in one deep mutational scan and got a usable reading for 93.4% of

them. In such an experiment every variant is mixed into one pool, and the pool goes through a selection step that keeps variants binding a target. A variant that binds better is kept in larger numbers. Each protein stays physically linked to the DNA that encodes it, so counting that DNA counts the protein. The DNA is counted before and after selection by sequencing, which reads DNA in short pieces. Each piece is one read, a variant's count is how many reads matched it, and the reads per variant is the *read depth*.

In that study a variant's fitness is its after-to-before read ratio, divided by the same ratio for the unmodified protein the study started from. It has nothing to do with `GDIF`, the grid's reference candidate. The study's published read totals average 557 reads per variant before selection and 285 after. Treating each count as Poisson, the variance of the log ratio is about $1/285 + 1/557$, so its standard deviation, the relative error, is about 7.3%. We apply this average depth to every candidate, which gives the constant 0.073. Real counts are often more spread out than Poisson, so this is a lower estimate of that study's counting error.

Taking the starting protein's ratio as the study's average, 285/557, a variant with 557 reads before selection and a single read after has fitness $(1/557)/(285/557)$, or 0.0035 units. That is the smallest non-zero value the study could resolve, and we use it as the smallest spread any reading has. It matters only near zero fitness, so it has no practical effect on a viable candidate.

A variant that binds better gets more reads, so one average depth overstates the counting error of the best candidates. A real re-run also varies for reasons beyond counting. The simulation may therefore be noisier or less noisy than a real lab, and which of the two holds is not known.

The reference candidate. `GDIF` is the viable candidate at the median fitness. Its true fitness, 6.8730, is stated to the agent to two decimals, and Table 12 gives the re-testing share with and without re-orders of it. We also checked that the scripted results do not depend on this start. On each landscape we drew 50 viable starts at random and ran the three scripted rules from each, on our campaign seeds. The guessing and informed rules keep `GDIF`'s best number of re-tests from every start on all three landscapes, and on the grid the guessing rule's re-testing share at that setting stays between 23.4% and 23.8%. On the grid the model-guided rule's best setting moves between no re-test and one a round, which score almost the same: at our setting of one re-test its share stays between 37.0% and 37.9%, and keeping that setting costs a median of 0.008 in final pick, at most 0.059. On GB1 and PhoQ it keeps `GDIF`'s setting from 49 and 50 of the 50 starts.

What the simulation leaves out. A non-viable order is caught and replaced. In a lab, finding out would cost a well. A new variant and a re-test cost the same slot. A lab must first make a new variant, which makes exploring dearer there and could raise a lab's best rate of re-testing. A lab usually wants a shortlist, and ranking a shortlist could reward more re-testing than one final pick does. Real rounds use plates of 96 or 384 wells, so the best number of repeats found here does not transfer as a number. Whether the agents' misallocation depends on batch size or budget is not tested. Noise here has no plate effects or drift, meaning slow shifts in readings between plates or over time, and in a real lab a repeat can also catch those. So an agent that re-tests more than the scripted rules here would not necessarily re-test too much in a real lab.

B The scripted rules

The guessing and informed rules of Section 2 were the prompt study's references, and the model-guided rule was added later under a registered change. The guessing and informed rules spend k of

the four slots a round re-testing their current best, and k^* is the best setting. The informed rule is told which candidates are viable. It is never rejected, so it loses no slots to rejections, and it still has to find out how good each candidate is. The agents are not told which candidates are viable, so the informed rule has an easier task than theirs. The guessing rule was pre-registered. It tries every candidate one letter away from its current best, and follows the agents' rules for rejections, replacements and lost slots, in the same code. Both ran on the same 1,500 seeds per setting.

The model-guided rule spends its first four rounds trying one-letter changes of the start, one at each position. From the fifth round it orders three new candidates a round, ranked by its letter scores, and re-tests one leading candidate: its best, or the next best once the best has two readings. In its last three rounds it re-tests its four best candidates by mean reading. It picks with the registered selection rule of Section 3.3.

Table 2: The guessing and informed rules on the grid. One knows more than the agents, the other knows the same.

	informed (told which are viable)	guessing (must guess)
best re-tests a round, k^*	1 of 4 slots	1 of 4 slots
mean pick at k=0	7.8862	7.7098
mean pick at k^*	7.9993	7.8659
distinct candidates at k^*	39.14	28.94
rejections per campaign at k=1	0.00	83.02
lost slots per campaign at k=1	0.00	11.70
campaigns collapsing at k=4	16 of 1,500	147 of 1,500

On the grid both do best at one re-test a round. Every comparison between the agents and the scripted rules there reaches the same conclusion with either rule.

C Using Wellocate

Interface. An episode is one campaign. A call to `step` orders up to four candidates, or one replacement after a rejection, and `nominate` ends the episode. Observations hold readings, outcomes and the remaining budget, and no true value; the reward is the nominee's true value. A text mode returns the exact prompts our agents received and reads replies the same way, asking again when a reply cannot be read. A language model and a policy trained by reinforcement learning thus face the same task. The environment runs the same code as our experiments. Through it the guessing rule reproduces our reference pick of 7.8659 and re-testing share of 23.78% exactly, the informed rule its pick of 7.9993, and one core runs about 159 episodes a second.

Tracks and splits. The protein track uses scrambled GB1 and PhoQ, with fresh scrambles for training, validation and testing and the real amino-acid names held out (Table 4). A scramble changes only the names, so a protein test split asks for unseen names of the same protein. Training on one protein and testing on the other asks for transfer. The grid track gives fresh NK landscapes and holds out rougher ones. Every split is fixed and public. A hidden test server is future work.

Scoring and baselines. The scorecard gives each policy's mean pick and its regret against the informed rule on the same episodes. It adds the re-testing share, the exploration and selection losses, the candidates measured and the slots lost, each with a 95% interval. Beside them it names the

Table 3: The guessing rule's final pick with no re-tests and with one re-test a round, with 95% intervals.

	no re-tests	one re-test a round
GB1, 8 times assay noise	3.60 [3.53, 3.67]	4.20 [4.11, 4.29]
grid, free rejections, assay noise	7.79 [7.77, 7.82]	7.95 [7.93, 7.97]

best scripted share for that landscape and noise. The baselines are random ordering, the guessing rule, the informed rule and the model-guided rule. A fifth, a simple untuned confirmation policy, explores without repeats and re-tests its top candidates at the end. The scorecard also lists our language-model results for comparison.

Noise and the best number of re-tests. On the grid the guessing rule that never re-tests loses 0.18, 0.36 and 0.54 at selection at half, one and two times assay noise. On PhoQ one re-test a round ties with no re-tests at about 4 times assay noise, and wins clearly at 8 times. On GB1 it wins only at 8 times (Table 3). Part of re-testing's value on the grid is avoiding rejections: the rule that never re-tests loses 23 slots to rejections, against 12 for one re-test a round. With free rejections, one re-test a round still beats no re-tests on the grid at assay noise (Table 3), so there re-testing also pays because of the noise. With free rejections the best number of re-tests also rises at lower noise on the grid, and GB1 and PhoQ keep the best settings of Figure 6 (Figure 7).

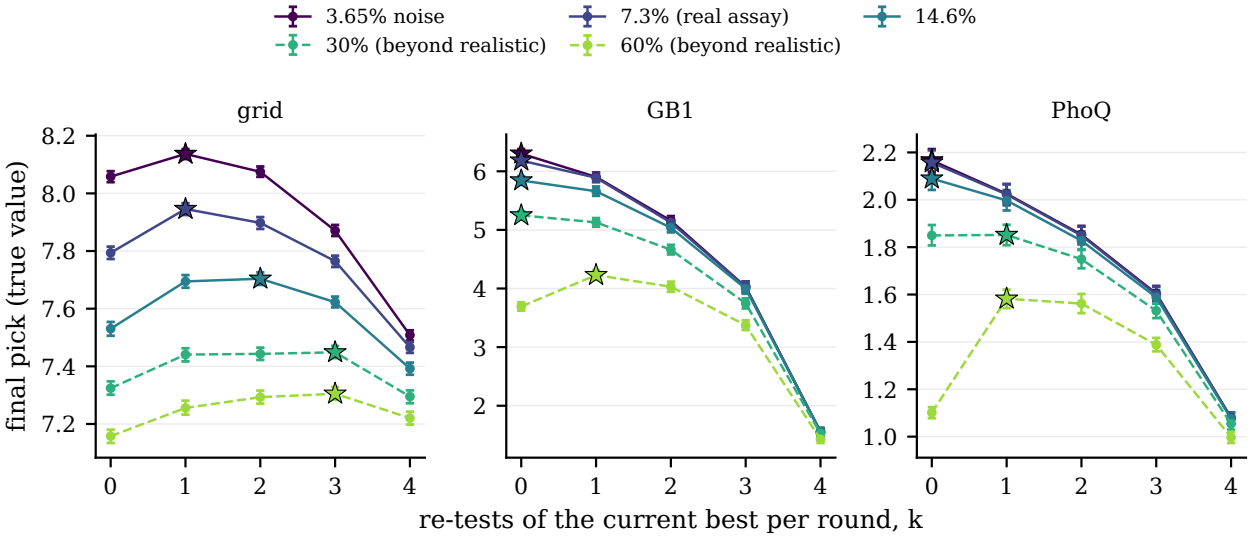


Figure 7: Figure 6 with free rejections: a rejected order never costs a slot.

Table 4: Wellocate's frozen splits: landscapes / episodes per split. Train, validation and test use disjoint landscape seeds (scrambles for GB1 and PhoQ, NK seeds for the grid) and disjoint campaign seeds. Held out: the real amino-acid names for GB1 and PhoQ, and two rougher NK settings, $K = 2$ and $K = 3$. *Paper* is the landscape and seeds used in this study.

track	landscape	train	validation	test	held out	paper
protein	scrambled GB1	400 / 100,000	10 / 1,500	20 / 1,500	1 / 1,500	1 / 1,500
protein	scrambled PhoQ	400 / 100,000	10 / 1,500	20 / 1,500	1 / 1,500	1 / 1,500
grid	generated NK	400 / 100,000	10 / 1,500	20 / 1,500	$K = 2$: 20 / 1,500 $K = 3$: 20 / 1,500	1 / 1,500

D What was registered

The prompt study's design, outcome, confirmatory contrasts and analysis were registered before it ran (Appendix H). Every later change was registered, with its date, before the data it governs. These changes added the aids experiment on the grid, with labels for a large, a small or no change in re-testing fixed in advance. They also added the landscape note, the model-guided rule and the Menu on scrambled GB1. Later changes added GPT-6.1 Sol and Gemini 3.1 Pro on the grid and on GB1, and the PhoQ runs for all five models with a named test. The last added the replication of the Decision-rule result, the learnability simulation and the test of the trained policy. A cap on repeat orders was also registered and then withdrawn before its main run: in trial runs the agents met the cap by proposing new names, most of them non-viable. No seed was dropped from the GPT-6.1 Sol, Gemini 3.1 Pro or PhoQ runs. Exclusions in other runs are reported where they occur. Every other analysis in these appendices was decided after the registered ones. It describes, checks or explores the results and tests no claim, and so does the noise sweep of Section 4.

E The five models

All five models ran at temperature 0 (Table 5). OpenRouter sends each call for an open-weight model to one of several hosting companies, so in every run DeepSeek V3.1's calls went to several hosts, and its results average over them. Opus 4.7, Gemini 2.5 Flash and DeepSeek V3.1 ran with reasoning off. GPT-6.1 Sol and Gemini 3.1 Pro ran at their lowest reasoning setting. Even so, Gemini 3.1 Pro used more than 16,000 reasoning tokens on each of 35 calls, up to 30,721 on one. All five saw the Blind prompt of Appendix H, which says nothing about measurement error or repeats.

Table 5: The five models as run on the grid. GPT-6.1 Sol and Gemini 3.1 Pro ran at their lowest reasoning setting. Reasoning tokens are the median over every logged call of the runs with the Blind prompt. Sol reported zero reasoning tokens on at least half of its calls.

model	released	reasoning as run	median reasoning tokens per call
Opus 4.7	2026-02	off	0
Gemini 2.5 Flash	2025-06	off	0
DeepSeek V3.1	2025-08	off	0
GPT-6.1 Sol	2026-09	minimal	0
Gemini 3.1 Pro	2026-02	minimal	240

Table 6: The five models on the grid, with 95% intervals. The second column counts re-testing over slots not lost.

	share (%)	share, slots not lost (%)	final pick	exploration loss	selection loss
Opus 4.7	74.9	77.4	7.621	1.02	0.12
	[73.5, 76.3]	[76.1, 78.7]	[7.543, 7.696]	[0.96, 1.09]	[0.08, 0.16]
Gemini 2.5 Flash	94.0	94.6	7.701	1.04	0.02
	[93.1, 94.8]	[93.8, 95.3]	[7.680, 7.720]	[1.03, 1.06]	[0.00, 0.03]
DeepSeek V3.1	91.0	91.2	7.760	0.94	0.06
	[90.3, 91.6]	[90.7, 91.8]	[7.716, 7.799]	[0.90, 0.98]	[0.04, 0.08]
GPT-6.1 Sol	44.1	45.7	7.672	0.93	0.16
	[41.5, 46.7]	[43.1, 48.4]	[7.582, 7.764]	[0.84, 1.01]	[0.11, 0.22]
Gemini 3.1 Pro	14.0	17.5	7.688	0.70	0.38
	[12.6, 15.3]	[15.9, 19.1]	[7.580, 7.802]	[0.60, 0.79]	[0.30, 0.46]

E.1 Results on the grid

Table 6 gives each model's re-testing share, final pick and losses. The scripted band runs from 23.78% to 37.61% of slots, or 29.80% to 44.10% counted over slots not lost, and the guessing rule's exploration and selection losses are 0.67 and 0.23. Every model's pick interval lies below the guessing rule's 7.87. Gemini 3.1 Pro's final pick had a single reading in 78% of its campaigns, against 2% for the informed rule and 2% for the guessing rule. With reasoning on (Appendix I), Opus 4.7 re-tests 69% of its slots and Gemini 2.5 Flash 82%. The aids' effects on re-testing are in Table 7.

Table 7: Change in re-testing share on the grid when each aid is added, in percentage points against the same model's control on the same seeds, with 95% intervals. Negative values mean less re-testing. The Menu's effect on scrambled GB1 is in Figure 5.

model	ledger	landscape note	Menu
Opus 4.7	+4.4 [+2.6, +6.2]	−4.7 [−7.8, −1.5]	−18.2 [−20.8, −15.5]
Gemini 2.5 Flash	−0.9 [−2.1, +0.2]	−0.5 [−1.8, +0.8]	−26.2 [−27.9, −24.5]
DeepSeek V3.1	+0.8 [+0.1, +1.6]	+1.5 [+0.8, +2.2]	−31.4 [−33.5, −29.4]

Each model's numbers come from the runs that its registered comparison with the scripted rules uses. Gemini 2.5 Flash and DeepSeek V3.1 come from the control runs of the aids experiment, whose prompt options match those of the GPT-6.1 Sol and Gemini 3.1 Pro runs. Opus 4.7 comes from a rerun of the original prompts made on the same day as prompt check A (Appendix H.6). That rerun differs only in the message sent back when a reply cannot be read. Counting re-testing over slots not lost leaves the conclusions unchanged. The GPT-6.1 Sol and Gemini 3.1 Pro runs cost $20.39 and $71.55, and the PhoQ runs $20.89. The scripted rules and Wellocate cost nothing to rerun.

E.2 Results on scrambled GB1

These runs used different landscapes and seeds from the grid runs, so they do not pair with them; Table 1 gives their shares. Gemini 2.5 Flash measures 3.9 distinct candidates and DeepSeek V3.1 2.4, both in the control runs of the GB1 Menu experiment. For Opus 4.7, the first 30 campaigns of an earlier control run measure 19.8 and re-test 61.7%; Table 1 uses all 60 campaigns of that run. Sol measures 45.8 and Gemini 3.1 Pro 46.9. On GB1 Gemini 3.1 Pro loses mostly through exploration, 3.76, against 0.10 at selection.

Table 8: Sol's pick minus the guessing rule's mean, with 95% intervals that include the guessing rule's own uncertainty: as registered, and under two checks that count its repeated nominee as one. The checks were not registered.

	GB1	PhoQ
registered: campaigns resampled by seed	+0.28 [+0.06, +0.53]	+0.32 [+0.10, +0.54]
whole nominees resampled	+0.28 [−0.01, +0.74]	+0.32 [−0.09, +0.55]
repeated nominee counted once	+0.33 [−0.01, +0.71]	+0.20 [−0.14, +0.57]
both checks together	–	+0.20 [−0.16, +0.61]

Table 9: Change with the Menu on the grid, against each model's control on the same seeds, with 95% intervals. For Opus 4.7 only the selection loss was computed.

	best value measured	selection loss
Gemini 2.5 Flash	+0.17 [+0.11, +0.23]	+0.13 [+0.10, +0.17]
DeepSeek V3.1	+0.08 [+0.01, +0.16]	+0.10 [+0.05, +0.14]
Opus 4.7	–	+0.09 [+0.03, +0.16]

E.3 Sol on scrambled GB1

Sol nominated the same candidate, true value 6.275, in 12 of its 30 GB1 campaigns without the Menu. Its registered lead over the guessing rule resamples campaigns by seed, so it counts those twelve campaigns as independent. Two checks that count them as one leave the estimate about the same and widen its interval to include zero (Table 8): one resamples whole nominees, and the other counts the repeated nominee once. Sol's lead on GB1 is therefore uncertain. The Menu's effect on its pick stays undetectable under both checks: −0.25 [−1.10, +0.41] and −0.29 [−1.14, +0.54].

E.4 How the Menu changed exploration and selection on the grid

On the grid the Menu raised the best value Gemini 2.5 Flash and DeepSeek V3.1 measured, and it worsened the selection of all three over-confirming models (Table 9). When a simple rule picks from the agents' own readings, the Menu improves the final pick of Opus 4.7 and Gemini 2.5 Flash. For DeepSeek V3.1 it makes no detectable difference.

F The five models on PhoQ

The PhoQ runs repeat the GB1 control runs, those without the Menu, on scrambled PhoQ, for all five models. Each model ran at the settings of its GB1 runs, and only the landscape changed. The scramble is our own and differs from all of Wellocate's frozen training, validation and test scrambles. Every model ran 30 campaigns on the same seeds, so the models pair by seed. The design, the outcomes and a named test were registered before the first call.

Table 10: The five models on scrambled PhoQ, with 95% intervals. The last three columns are each model's pick minus the guessing rule's: against its mean over 1,500 reference seeds (the registered comparison), and paired with the guessing rule on the model's own seeds, on PhoQ and on GB1.

			pick minus the guessing rule		
	share (%)	final pick	registered	paired, PhoQ	paired, GB1
Opus 4.7	62.8	1.893	−0.16	−0.26	−1.41
	[61.0, 64.6]	[1.753, 2.025]	[−0.30, −0.02]	[−0.60, +0.07]	[−1.92, −0.89]
Gemini 2.5 Flash	63.7	2.130	+0.08	−0.02	−5.07
	[60.6, 66.6]	[2.050, 2.184]	[−0.01, +0.15]	[−0.35, +0.28]	[−5.47, −4.68]
DeepSeek V3.1	89.9	0.829	−1.22	−1.32	−5.35
	[87.5, 91.9]	[0.757, 0.942]	[−1.31, −1.10]	[−1.68, −0.98]	[−5.75, −4.94]
GPT-6.1 Sol	23.9	2.368	+0.32	+0.22	−0.06
	[20.8, 27.3]	[2.155, 2.582]	[+0.10, +0.54]	[−0.21, +0.62]	[−0.65, +0.52]
Gemini 3.1 Pro	14.7	1.725	−0.33	−0.42	−1.52
	[13.1, 16.3]	[1.601, 1.857]	[−0.46, −0.19]	[−0.78, −0.10]	[−2.44, −0.55]

Re-testing on PhoQ. On GB1 every model re-tests more than the best scripted rule. The registered test extends that claim to PhoQ only if each model's 95% interval lies above PhoQ's best scripted share. On PhoQ that rule is the informed rule with no re-tests, so its share is zero. Every interval lies above it, and also above the guessing rule's 1.03% (Table 10). Gemini 2.5 Flash lost 3.7 slots a campaign to rounds it could not complete, and a lost slot is never a re-test.

Picks against the guessing rule. The registered comparison sets each pick against the guessing rule's mean over its 1,500 reference seeds, 2.050, and adds that rule's own uncertainty. A second, descriptive comparison runs the guessing rule on each model's own seeds and pairs the two picks seed by seed. Paired this way, DeepSeek V3.1 and Gemini 3.1 Pro pick worse than the guessing rule on PhoQ, and the other three are about level with it. On GB1 every model except Sol picks worse. In the registered comparison DeepSeek V3.1, Gemini 3.1 Pro and Opus 4.7 pick worse, Gemini 2.5 Flash is level and Sol is better.

Sol's lead on PhoQ. Sol nominated one candidate, IAQT, in 14 of its campaigns, among 13 different nominees. Its registered lead therefore rests partly on that one candidate. Under the checks of Appendix E.3 every interval includes zero (Table 8), so Sol's lead on PhoQ is uncertain too.

A replication on other seeds. An earlier run put the three earlier models on the same PhoQ landscape, under the same Blind prompt, on other seeds. Its shares were 65.3% [63.0, 67.6] for Opus 4.7, 61.7% [58.7, 64.5] for Gemini 2.5 Flash and 91.4% [90.3, 92.3] for DeepSeek V3.1, close to the shares above. Its picks were 1.89, 2.12 and 0.81, close to those above.

G The confirmation simulation

Rebuilding each campaign. The nomination prompt holds a campaign's whole history, one line per order. Replaying those orders through the simulated assay with the campaign's seed reproduces every reading. The noise and the failures are fixed by the seed, the candidate and its order count. A campaign was used only if every logged outcome and reading was reproduced and the rebuilt registered pick and re-test count matched the original run's. We rebuilt the grid campaigns of Gemini

3.1 Pro and Sol, and as a check those of Gemini 2.5 Flash and DeepSeek V3.1. Every campaign passed.

Replacing the last round. With the last round replaced, Gemini 3.1 Pro gains against its own pick at $m = 3$. It gains about as much at $m = 4$, where the interval just includes zero. At $m = 3$ its re-testing share rises to 18.9% (Table 11). The registered rule applied to its unchanged readings gains almost nothing, so the gain comes from the extra readings. Gemini 2.5 Flash and DeepSeek V3.1 gain a little at $m = 1$. The simulation drops the agent's own final-round orders, so it gives up whatever those would have found.

Table 11: Change in final pick against each model's own pick in the confirmation simulation, with 95% intervals.

model	replay	change in final pick
Gemini 3.1 Pro	last round, $m = 3$	+0.088 [+0.011, +0.171]
Gemini 3.1 Pro	last round, $m = 4$	+0.090 [−0.004, +0.190]
Gemini 3.1 Pro	registered rule on unchanged readings	+0.004 [−0.028, +0.036]
Gemini 2.5 Flash	last round, $m = 1$	+0.014 [+0.002, +0.032]
DeepSeek V3.1	last round, $m = 1$	+0.035 [+0.016, +0.055]
Gemini 3.1 Pro	last two rounds, $m = 3$	+0.100 [+0.013, +0.191]
GPT-6.1 Sol	last two rounds, $m = 1$	−0.041 [−0.100, +0.011]

The last two rounds. Replacing the last two rounds, with the top m chosen after the round before them, recovers about 27% of Gemini 3.1 Pro's selection loss at $m = 3$, close to one round's 23% at the same m. Sol's pick falls slightly at $m = 1$, with an interval that includes zero. This run was not planned in advance.

H The pre-registered prompt study

This appendix reports the pre-registered prompt study: three models under five prompt conditions on the grid, with its prompt checks, its zero-noise run and its independent check. It is the evidence behind the over-confirmation in Section 3.1. Table 12 compares each cell with the guessing rule on how good the best candidate it tested was and on how well it picked among what it tested.

H.1 Setup

The grid uses the generated landscape, campaign, noise and failures of Section 2 and Appendix A. The agent is told only that it is running a numerical optimisation campaign, and never that it is working on proteins.

Why the landscape is generated. A generated landscape makes the true value of every candidate known, and no model can have seen its best candidate. A published landscape cannot guarantee either. Given the GB1 protein landscape of Wu et al. [2016] with its real amino-acid letters, Opus 4.7 named the best measured variant in round one in 11 of 20 campaigns. Four random viable picks would do so 0.035% of the time. The prompt study uses the grid only, and the five-model study adds GB1 and PhoQ.

Five prompt conditions. Five conditions vary what the agent is told, each adding to the one before. **Blind** says nothing about measurement error. **Quantitative** adds that readings carry error and gives $\sigma(f)$. **Mechanism** adds that averaging n readings reduces the standard error by $\sqrt{n}$, and permits re-ordering, without recommending it. **Aggregated** keeps Mechanism's text and changes the environment: the history is shown as a count, mean and standard error per candidate, in place of one line per reading. **Decision-rule** adds the selection rule outright. No condition ever instructs the agent to re-test, because that would measure instruction-following.

Models and scale. Three models, called Opus 4.7, Gemini 2.5 Flash and DeepSeek V3.1 throughout this paper and requested from OpenRouter as `anthropic/claude-opus-4.7`, `google/gemini-2.5-flash` and `deepseek/deepseek-chat-v3.1`, at temperature 0. We call one model under one prompt condition a *cell*, so the prompt study has 15 cells. It holds 5,075 campaigns and 162,986 model calls. Cells are sized per model: 695, 139 and 181 campaigns per cell.

Pre-registration. The design, the outcome, the six confirmatory contrasts and the analysis were fixed before the prompt study ran (Appendix D). The six contrasts are Aggregated against Mechanism and Decision-rule against Mechanism, for each model, with Holm correction for multiple comparisons across the six. Contrasts are paired by seed. The seed fixes measurement noise and failures, so at a given seed the k-th reading of a candidate is the same in every condition.

The scripted rules. The guessing and informed rules of Appendix B are the grid's references. Both do best at $k^* = 1$, one slot in four.

H.2 Re-testing share

Agents spend most of their measurements re-testing candidates they have already measured. Across the 15 cells the re-testing share runs from 75% to 96% of used slots; both scripted rules spend a quarter (Table 12, Figure 8). The count includes re-orders of the reference candidate, whose score the agent is given at the start. On this count every cell re-tests more than a scripted protocol that measures each candidate three times. Without those re-orders the share is 57% to 75%, and six cells fall below that protocol. Counting only slots spent on a candidate that already has a successful reading, which leaves out re-orders after a failed order, gives 74.7% to 95.1%, within 1.4 points of the headline in every cell.

The scripted lab protocols. Three scripted protocols spend the same 60 slots the way a wet lab is often run, searching around their current best as the scripted rules do. Triplicate-everything measures 20 candidates three times, 66.7% of slots on repeats. Duplicate-everything measures 30 twice, 50.0%. Screen-then-confirm takes 48 single readings and then confirms the leaders, 20.0%. These protocols are simulated, and a study with human experimenters is future work.

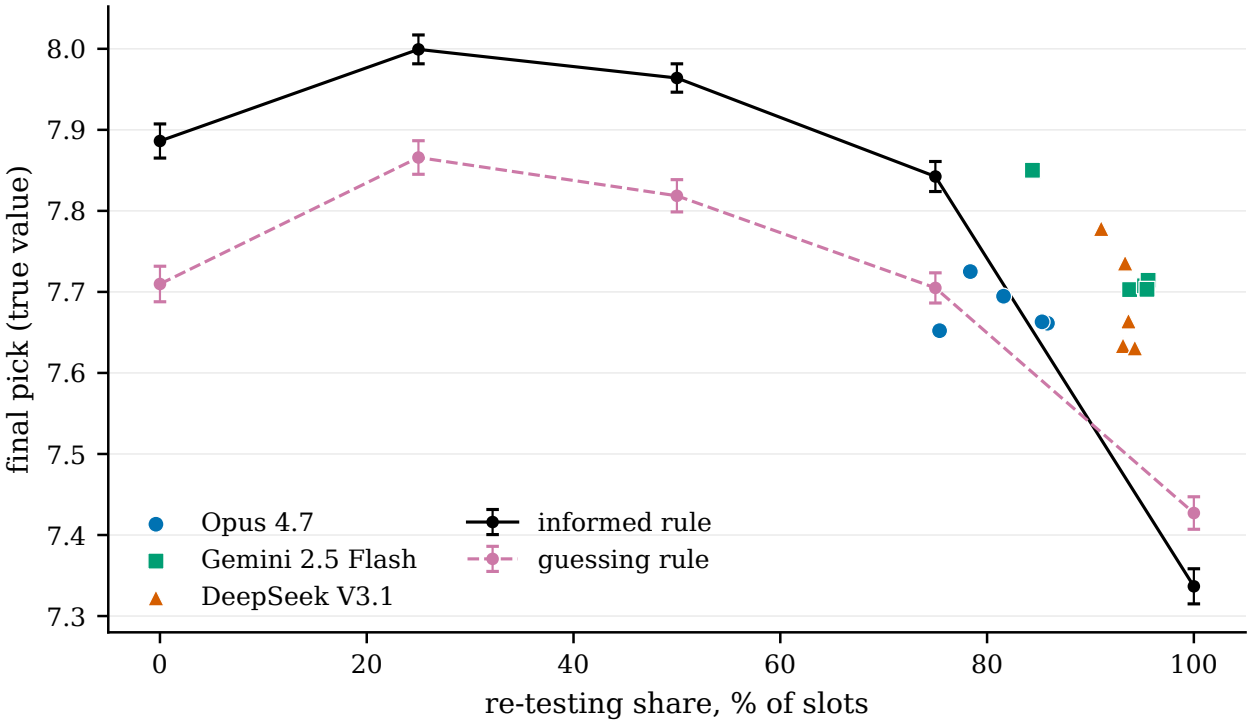


Figure 8: True value of the final pick against the re-testing share, one point per cell. The two lines are the informed rule (upper, told which candidates are viable) and the guessing rule (lower), spending $k = 0$ to 4 of the four slots each round re-testing their current best, with 95% intervals.

The re-tests pay off in selection. Agents picked well among the candidates they had measured, closer to the best of what they tested than the guessing rule. The best candidate they tested was much worse than that rule's best, because they tested so few (Figure 9). In every cell the shortfall from testing too few outweighs the gain from picking well, so the agents end up behind (Table 12, Figure 11; Appendix H.7). Their stated reasons show that they climb from their current best one letter at a time, as the scripted rules do, in lines such as *"vary pos4, C works so try neighbor"*. They climb a few steps and then spend most of the budget confirming the same few candidates, so the climb stalls.

H.3 Exact readings

Removing the noise lowers re-testing in every cell, significantly in 12 of 15. Re-testing still takes roughly half to nearly all of the slots, and with exact readings no re-test is needed. The zero-noise run sets σ to zero and changes nothing else, so the agent is not told its readings are exact; Appendix H.4 tests the case where it is told. Here re-testing is counted after the first repeat an agent sees, as the share of slots spent on a candidate that already has a successful reading. This is a stricter count than the headline share, and it is compared with the same count under noise at the same seeds (Figure 10).

At $\sigma = 0$, re-testing after the first repeat runs from 51.8% to 97.5% across the 15 cells, with a median of 90.6%. The lowest cell is Quantitative with Opus 4.7, at 51.8%, 95% interval [48.2%, 55.4%]. The highest is Aggregated with DeepSeek V3.1, at 97.5%, [96.7%, 98.3%]. By model, Opus 4.7 runs 51.8% to 71.0%, Gemini 2.5 Flash 76.8% to 94.4% and DeepSeek V3.1 94.1% to 97.5%. All 15 paired differences from the same cells with noise are negative, from −35.0 to −0.9 percentage

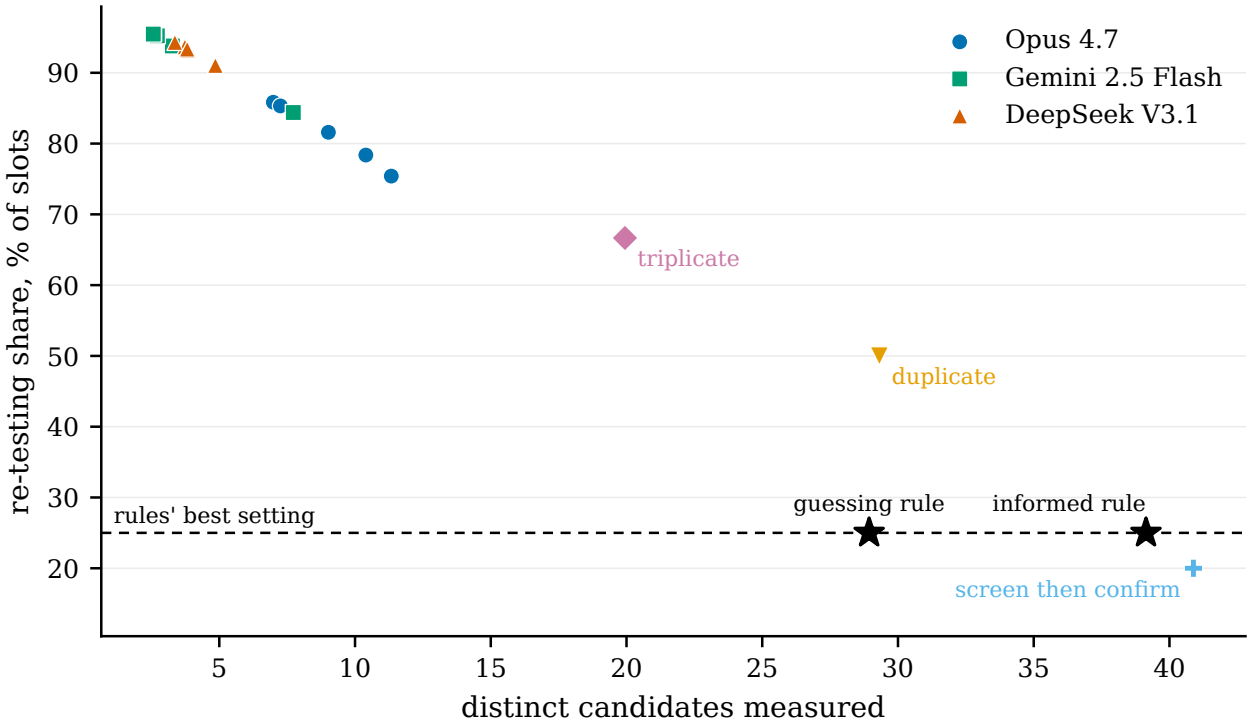


Figure 9: Re-testing share against distinct candidates measured, one point per cell, with the informed and guessing rules at their best setting and the three scripted lab protocols.

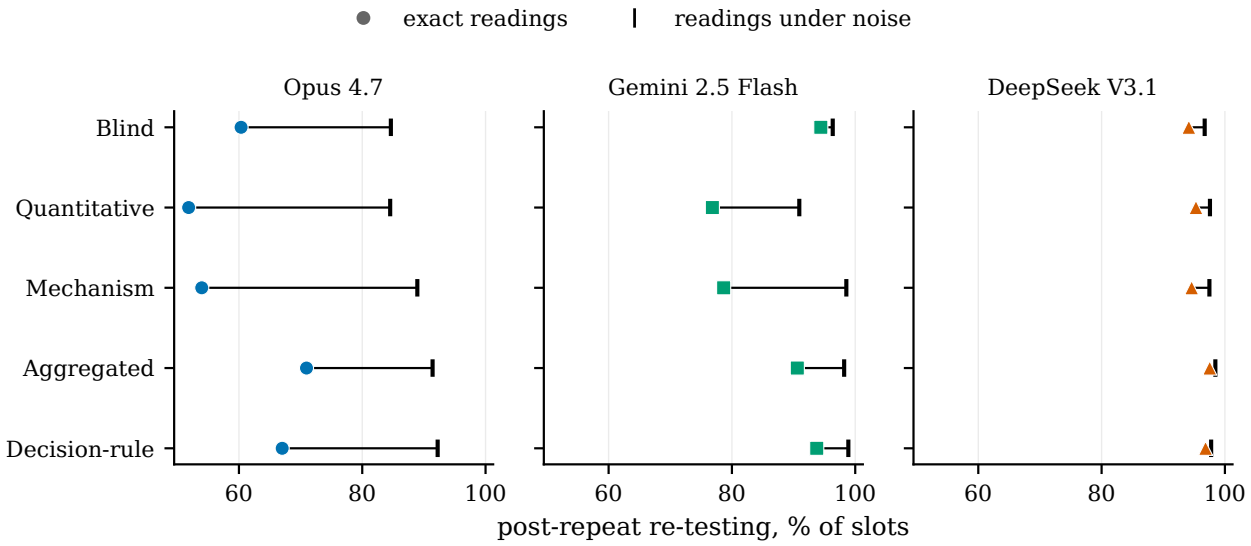


Figure 10: Re-testing after the first repeat when every reading is exact (the marker) and the level implied under noise (the bar), one row per cell. A repeat of an exactly known value returns no new information.

Table 12: Results for each cell, one model under one prompt condition. *Share* is the re-testing share, the fraction of used slots spent re-testing a candidate already ordered. *Excl. start* leaves out re-orders of the start, whose reading the agent is given in advance. *Distinct* is the mean number of different candidates measured per campaign, and *pick* the mean true value of the final pick. The last three columns compare each cell with the guessing rule at its best setting, which measures 28.94 distinct candidates and reaches 7.87. *Tested* is how much further the best candidate the agents measured falls short of the global best. *Picked* is the change in how far their pick falls short of the best they measured, negative where they picked better. *Net* is the sum, the agents' shortfall in the true value of the final pick; rounded parts may not add to it exactly.

		share (%)				against the guessing rule		
model	condition	all	excl. start	distinct	pick	tested	picked	net
Opus 4.7	Blind	75.4	62.8	11.3	7.65	+0.33	−0.12	+0.21
	Quantitative	78.4	65.5	10.4	7.73	+0.25	−0.11	+0.14
	Mechanism	81.6	68.1	9.0	7.69	+0.31	−0.14	+0.17
	Aggregated	85.8	70.4	7.0	7.66	+0.37	−0.17	+0.20
	Decision-rule	85.3	70.8	7.3	7.66	+0.36	−0.16	+0.20
Gemini 2.5 Flash	Blind	93.8	69.5	3.3	7.70	+0.37	−0.21	+0.16
	Quantitative	84.4	67.3	7.7	7.85	+0.18	−0.16	+0.02
	Mechanism	95.2	73.3	2.7	7.71	+0.38	−0.22	+0.16
	Aggregated	95.6	74.6	2.6	7.71	+0.37	−0.22	+0.15
	Decision-rule	95.5	73.8	2.6	7.70	+0.38	−0.21	+0.16
DeepSeek V3.1	Blind	91.0	68.6	4.9	7.78	+0.26	−0.17	+0.09
	Quantitative	93.1	63.1	3.9	7.63	+0.41	−0.18	+0.23
	Mechanism	93.7	56.9	3.7	7.66	+0.38	−0.18	+0.20
	Aggregated	94.3	57.0	3.4	7.63	+0.43	−0.19	+0.24
	Decision-rule	93.4	58.8	3.8	7.74	+0.31	−0.18	+0.13

points, and 12 of 15 are significant at 0.05 after Holm correction within that run. Every cell pairs all 40 seeds. The size of the drop tracks the level a model starts from. Opus 4.7 falls furthest. DeepSeek V3.1 falls least, from levels of 96.7% to 98.5% under noise. Elsewhere in these appendices, zero-noise numbers count all used slots.

H.4 Independent check

In a separate one-round test with exact readings, the models still spent from about a quarter to over half of the round re-testing, and adding noise raised re-testing sharply, so the models do respond to noise. The test was written separately from Wellocate, with its own world generator, prompt and scorer, and shares no code with it.

Each call shows one fixed round of an optimisation task. Every candidate is valid, so nothing is ever rejected. A seven-round history is already measured, and the best candidate has already been measured twice. Three versions vary only what is true and what is said about measurement error: exact readings with exactness stated, exact readings with nothing said, and noisy readings with the noise stated. Thirty paired histories per version, three models, temperature 0, 270 calls. Each reply is scored over four slots. A missing line counts as a new candidate, and so does a name with letters outside the alphabet.

A reading rule was fixed before any call. A repeat share of 25% or more in the exact-stated version confirms the behaviour; 5% or less attributes it to our environment; anything between is partial. Gemini 2.5 Flash and DeepSeek V3.1 clear the 25% line. Opus 4.7 at 24.2% falls just below

Table 13: Share of the four chosen measurements that repeat an already-measured candidate.

model	exact, stated	exact, unstated	noisy, stated	noise effect (paired)
Opus 4.7	24.2%	24.2%	61.7%	+37.5 pp (SE 5.3)
Gemini 2.5 Flash	45.0%	50.8%	95.0%	+50.0 pp (SE 7.0)
DeepSeek V3.1	55.8%	49.2%	94.2%	+38.3 pp (SE 8.3)

Table 14: The six confirmatory contrasts on the final pick's true value, paired by seed, with Holm correction across the family. The primary p comes from a bootstrap that resamples seeds (studentised wild cluster bootstrap, 20,000 draws). The intervals are unadjusted and the Holm p adjusts for the family of six. Seed counts differ because each model's cells were sized separately (Section H.1). †Did not hold up on 120 new seeds: +0.008, 95% bootstrap interval $[-0.036, +0.052]$ (Appendix J).

model	contrast	seeds	diff	95% CI	Holm p
Opus 4.7	Aggregated	695	−0.0334	$[-0.0606, -0.0061]$	0.0873
Opus 4.7	Decision-rule	695	−0.0316	$[-0.0583, -0.0049]$	0.0873
DeepSeek V3.1	Aggregated	181	−0.0332	$[-0.0757, +0.0092]$	0.3721
DeepSeek V3.1	Decision-rule	180	+0.0699	$[+0.0271, +0.1126]$	**0.0075**†
Gemini 2.5 Flash	Aggregated	139	+0.0067	$[-0.0130, +0.0265]$	1.0000
Gemini 2.5 Flash	Decision-rule	139	−0.0044	$[-0.0278, +0.0189]$	1.0000

it, so two models give a clear reproduction and one is partial.

Stating that readings are exact made no detectable difference. The paired difference between saying nothing and stating it is +0.0 pp (SE 2.4) for Opus 4.7, +5.8 (SE 7.1) for Gemini 2.5 Flash and −6.7 (SE 6.9) for DeepSeek V3.1. In the zero-noise run the noise effect looks small for Gemini 2.5 Flash and DeepSeek V3.1 because their re-testing is already high with exact readings, at 81.6% and 89.6% of all used slots against 51.4% for Opus 4.7, each a mean over the five conditions.

H.5 Pre-registered contrasts

If the behaviour came from a history that is hard to read, or from a missing rule, fixing those should reduce it. Six contrasts were fixed before the prompt study ran, comparing Aggregated and Decision-rule with Mechanism for each model, with Holm correction across the six (Table 14). One survives the correction: giving the selection rule outright improved the final pick for DeepSeek V3.1, by +0.0699. It did not hold up: a pre-registered replication on 120 new seeds gave +0.008 $[-0.036, +0.052]$ (Appendix J). In the original run the gain came through what DeepSeek V3.1 measured. Against the guessing rule, its shortfall in the best candidate tested fell from 0.38 to 0.31, and its selection gain stayed at 0.18: it picked among what it tested as well as before (Table 12). It picked the highest-average candidate about as often with the rule as without it (86% against 90%; an exact McNemar test for paired choices gives $p = 0.36$). Its re-testing share moved by less than half a point (93.7% against 93.4%), and it tried about as many candidates (3.7 against 3.8). Opus 4.7 responded differently. The rule raised its highest-average picks from 74% to 86% ($p < 0.001$), and it also re-tested more and tried about two fewer candidates. Opus 4.7 was nominally worse under both interventions; its unadjusted intervals exclude zero, and both differences lose significance after correction for the six contrasts. Gemini 2.5 Flash showed no detectable change. Arithmetic alone does not explain the habit either: the Aggregated condition shows each candidate's count, mean and standard error already computed, and re-testing was slightly higher under it than under Mechanism for all three models. We do not read any of this as evidence that stating a decision rule helps agents.

Table 15: Re-testing share in the prompt checks, as a percentage of used slots, on the same 60 seeds in every column. *Grid* is the original campaigns on those seeds. Check A states that the final pick is scored on its true value and corrects the replacement prompt's order count. Check B also makes a rejected order free. *Same-day rerun minus grid* is the change, in points, for the original prompts run again on the same day as check A.

model	condition	grid	check A	check B	same-day rerun minus grid
Opus 4.7	Blind	76.0	76.6	74.8	−1.1
	Mechanism	82.1	81.1	80.9	+0.2
Gemini 2.5 Flash	Blind	94.4	89.8	83.3	−0.1
	Mechanism	95.2	95.1	89.0	−0.9
DeepSeek V3.1	Blind	90.8	90.4	90.0	−0.9
	Mechanism	93.6	93.6	93.5	+0.1

H.6 Prompt checks

A review of the prompts found two flaws. The round and replacement prompts did not say that the final pick is scored on its true value. The replacement prompt also stated one order per round, where a round has four. A wrong guess about viability could cost a slot, so rejection risk might also push agents toward re-testing candidates known to work.

Three prompt changes were tested, each absent from the original prompts. The *goal* option adds one sentence to the goal in the round and replacement prompts, the same in all five conditions: "At the end you will nominate one candidate, and your score is that candidate's true value." It says nothing about readings, because the Blind condition withholds that readings carry error. The nomination prompt was left as it was, since it already asks for the highest true score. The *budget* option makes the replacement prompt's budget line read "15 rounds, 4 orders per round, 60 measurements in total." It had stated one order per round.

The *free-rejection* option makes a rejected order free. In every prompt, the sentence on replacements and lost slots becomes "An order for a non-viable candidate is rejected at no cost and you choose again. No slot is ever lost to a non-viable order." The agent may then be asked again up to 20 times for one slot. A slot that reaches that cap is lost and counted, and no slot in check B reached it. The guessing rule reached it in 353 of its 1,500 campaigns at one re-test per round, and still lost 0.86 slots per campaign.

Check A used the goal and budget options and check B all three, each on Blind and Mechanism for all three models, on 60 seeds, each paired with the original campaign on the same seed (Table 15). A rerun of the original prompts ran on the same day as check A. The registered statement for check B reads "rejection risk does not produce the habit". Intervals are 95% percentile bootstraps over the paired campaigns.

Check A fixes both flaws. With it, the re-testing share moved by less than 1.1 points in five of six cells. For Gemini 2.5 Flash under Blind it fell by 4.6 points [−6.6, −2.6], and under Mechanism Gemini's final pick fell by 0.160 [−0.256, −0.070]. The registered margin for final picks, ±0.06, could not be met with 60 pairs in four cells even with no true change. The re-testing share is therefore the informative outcome. The same-day rerun of the original prompts moved the re-testing share by at most 1.1 points from the original campaigns in every cell. The two Gemini shifts remain against this control, so they come from the wording.

In check B every cell still spent 74.8% or more of its slots on re-tests, against 23.96% for the guessing rule at its best setting under the same rule. Gemini 2.5 Flash re-tested 6 to 11 points less and picked better, and it stayed far above the guessing rule. The final pick of Opus 4.7 under

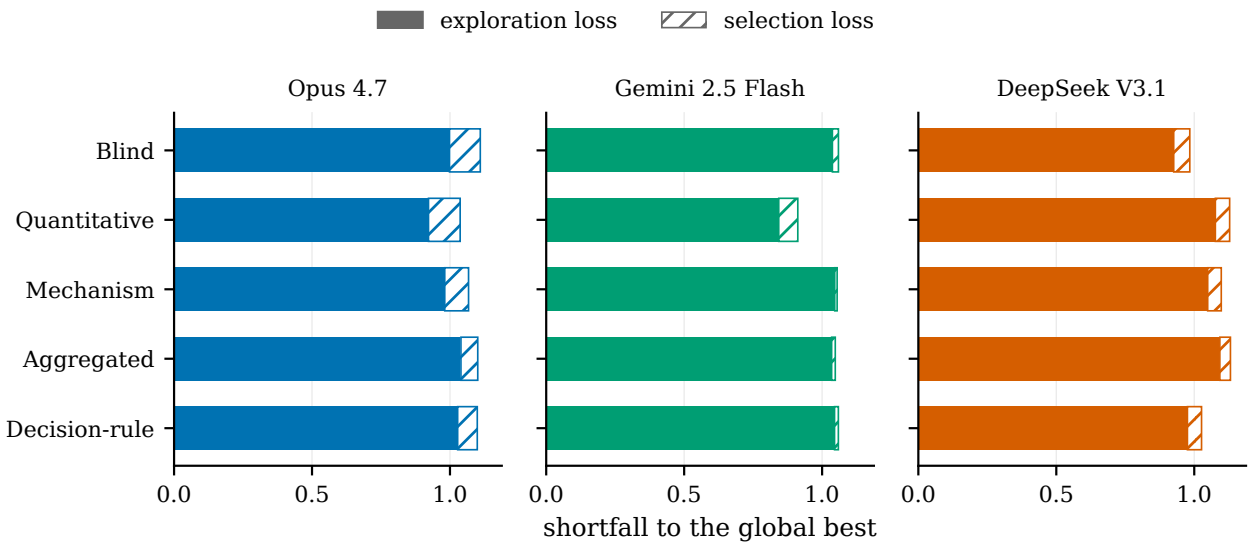


Figure 11: The shortfall to the global best in each cell, split into exploration loss and selection loss, which sum to the whole. The guessing rule's own split, 0.67 to exploration and 0.23 to selection, is the reference point of Figure 4.

Mechanism fell by 0.1340 [−0.2324, −0.0347].

H.7 Exploration, final picks and rejections

Prompt conditions and exploration. The explanatory conditions raise re-testing and cut exploration. For Opus 4.7, distinct candidates fall from 11.34 under Blind to 9.02 under Mechanism and 6.99 under Aggregated, and its re-testing share rises from 75.4% to 81.6% and then 85.8%. This is descriptive across conditions and is not one of the pre-registered contrasts.

Final picks against the scripted rules. Cell means of the final pick's true value run from 7.6308 to 7.8501. At k^* the guessing rule reaches 7.8659 and the informed rule 7.9993. Against the scripted rules with no re-tests, the answer depends on the rule. Five of the 15 cells sit above the guessing rule's k=0 mean of 7.7098, two with intervals entirely above, and every cell sits below the informed rule's 7.8862. The most exploratory cell reaches 39.2% of the guessing rule's count of distinct candidates. Split against the global best, exploration accounts for 0.8428 to 1.0928 of the shortfall and selection for 0.0087 to 0.1148, so in every cell exploration is 88.9% to 99.2% of the total and selection 0.8% to 11.1% (Figure 11).

Rejections. Cell means run 2.57 to 11.34 distinct candidates, against 28.94 for the guessing rule and 39.14 for the informed rule. Over all 5,075 prompt-study campaigns the agents were rejected 16.75 times per campaign and lost 0.71 slots, against 83.02 and 11.70 for the guessing rule under the same rules. The agents were rejected less often per campaign because they ordered far fewer new names: most of their slots went on re-tests. Per new name they were rejected 72.2% of the time, against 74.4% for the guessing rule. New names are rejected orders plus candidates measured, leaving out the starting candidate both are given. So the agents pick viable names about as well as the guessing rule's neighbour sampling. Across the 15 cells the rate runs from 68.5% to 89.0%. Eight of the 15 cells are rejected more often than the guessing rule and seven less often. The eight

are Gemini 2.5 Flash in every condition except Quantitative, and DeepSeek V3.1 in every condition except Decision-rule.

I Temperature, reasoning and a biology framing

Temperature. Three models under Mechanism at temperature 1.0, on the grid's own seeds. Re-testing moves by −0.3 percentage points (SE 0.8) for Opus 4.7 and −1.0 (SE 0.3) for DeepSeek V3.1, and falls 5.5 (SE 1.0) for Gemini 2.5 Flash, which leaves it at 89.9% of slots. The final pick moves by at most 0.07 in either direction, with standard errors of 0.04 to 0.06.

Reasoning. This experiment runs two of the five conditions on two of the three models, at reasoning effort medium, paired by seed against the prompt study with 35 to 40 seeds per cell after exclusions. Holm correction runs across its four cells only.

Reasoning shifted agents from re-testing toward exploration and their picks worsened, mainly because they found worse candidates. The re-testing share falls by 7.5 to 12.4 percentage points and distinct candidates rise by 3.7 to 7.1 per campaign in all four cells. The final pick's true value falls by 0.099 to 0.263, and all four cells reject at 0.05 after Holm. The true value of the best candidate the agent measured drops from 7.6854–7.7387 in the prompt study to 7.5302–7.6253 under reasoning, so exploration loss rises by +0.06 to +0.19, and selection loss by only +0.007 to +0.085. In 9 of 80 Gemini 2.5 Flash reasoning campaigns the model returned no final answer, eight of them at the 6,000-token output limit, and these are excluded. Keeping the nine excluded campaigns leaves the comparison unchanged: with all 40 paired seeds the differences are −0.0987, −0.2528, −0.2016 and −0.2338, and all four still reject.

Biology framing. Each of the three models ran three cells of 40 campaigns on the Blind prompt, each with one added sentence that gives the agent a wet-lab role. The cells hold the role alone, the role with exact readings, and the role with one more sentence saying readings are noisy. The result covers only what one sentence of lab context does to the behaviour. The candidates are four letters from A to T. Seventeen of the twenty letters are amino-acid codes. The landscape is generated, so a biochemical intuition the framing evokes has nothing to attach to.

The role lowers re-testing in all three models against the same prompt without it: −6.5 percentage points for Opus 4.7 [−8.5, −4.5], −9.8 for Gemini 2.5 Flash [−12.7, −7.0] and −2.9 for DeepSeek V3.1 [−4.7, −1.2]. Re-testing persists under the role with exact readings, at 49.1% to 86.4% of slots. The same persistence appears with no role at all: the zero-noise run's own Blind cells, which carry the prompt these role cells are built on, sit at 52.6%, 92.9% and 88.0% of all used slots. Within this experiment, exact readings lower re-testing against the role cell by 19.8, 8.3 and 1.8 percentage points, paired by seed, with DeepSeek V3.1's interval including zero. Saying that readings are noisy raised re-testing a little against the role cell, in all three models.

J A pre-registered replication of the Decision-rule result

One confirmatory contrast survived Holm correction: DeepSeek V3.1's final picks were better under Decision-rule than under Mechanism. We ran that comparison again on 120 seeds that no earlier run had used. The design, the outcome and the reading rule were fixed before any call. The landscape, the prompts and the settings were those of the prompt study.

The outcome is the paired difference in the final pick's true value, Decision-rule minus Mechanism. One Decision-rule campaign whose nomination did not parse is excluded, as in the prompt study's analysis, which leaves 119 pairs. The rule, fixed in advance, has three readings. A positive mean whose 95% bootstrap interval excludes zero replicates. A positive mean whose interval includes zero reads as "direction holds, not significant". A mean at or below zero does not replicate.

The mean difference is +0.008, with a 95% bootstrap interval of $[-0.036, +0.052]$. By the rule, the direction holds and the difference is not significant. Of the 119 pairs, 37 improved, 33 got worse and 49 were unchanged. The interval's upper end lies below the original estimate of +0.0699 and below 0.06, the smallest effect the study was designed to detect.

Not pre-registered: because the landscape and settings are the same, the replication's conditions can be set against the original run's. The Decision-rule means match: 7.736 originally and 7.736 in the replication. The Mechanism mean rose from 7.664 to 7.728, a difference of +0.064 (SE 0.023). This is consistent with the original gap coming mostly from a Mechanism sample that happened to score low.

K The learnability simulation

This simulation asks whether the best number of re-tests can be learned from the environment's reward alone. It makes no model calls. It runs the informed and the guessing rule at $K \in \{1, 2, 3\}$ and at noise of 0.5, 1 and 2 times ours, 18 settings in all. Each uses 200 fresh landscapes, one per seed, and training and held-out landscapes come from separate seed ranges.

Trained on the environment's reward alone, a two-parameter learner that chooses the number of re-tests and the selection rule settles on $k = 1$ with the registered selection rule of Section 3.3. That is k^* or tied with k^* in all 18 simulated settings. Sweeping k gives $k^* = 1$ in 13 settings, $k^* = 2$ in 3 and $k^* = 0$ in 2. In 16 of the 18 the gap between k^* and its runner-up sits inside two standard errors, so most settings are ties. Raw paired differences between the policy at k^* with the registered selection rule and the rule that never re-tests run −0.0077 to +0.2231 on the held-out landscapes, and both negative values are at half noise.

The right rate depends on the noise. It is zero when readings are exact. In the simulation one repeat per round is best or tied in all 18 settings. At half noise, never re-testing ties with one repeat in two settings at 200 landscapes, and at 1,500 landscapes the best is never zero.

The registered score and collapsed campaigns. The pre-registered score rescales each result between those of two scripted endpoints. Across landscapes those endpoints move by more than the gap between them, so a held-out score rescaled with training endpoints is unstable: 0.122 to 7.765 across the 16 cells where it is defined. Rescored with endpoints from the same held-out landscapes it runs 0.784 to 1.000. It is exactly 1.000 in the 11 cells where $k^* = 1$ and the score is defined, because the learner selects that policy, and undefined in four cells at half noise, where the held-out span is zero or negative. No claim in this paper depends on it. The guessing rule's means at high re-testing are over surviving campaigns, since a campaign collapses when fewer than two candidates are measured: at $K = 3$, 138 of 200 campaigns collapse at $k = 4$ and 12 of 200 at $k = 3$.

The larger run. At 1,500 landscapes per setting the picture holds: one re-test a round is best or tied in all 18 settings. The three settings where two came out ahead are all at double noise and all within two standard errors. Rescored with held-out endpoints, the learner selects one re-test a round with the registered selection rule in every setting. It scores exactly 1.000 in the 15 settings

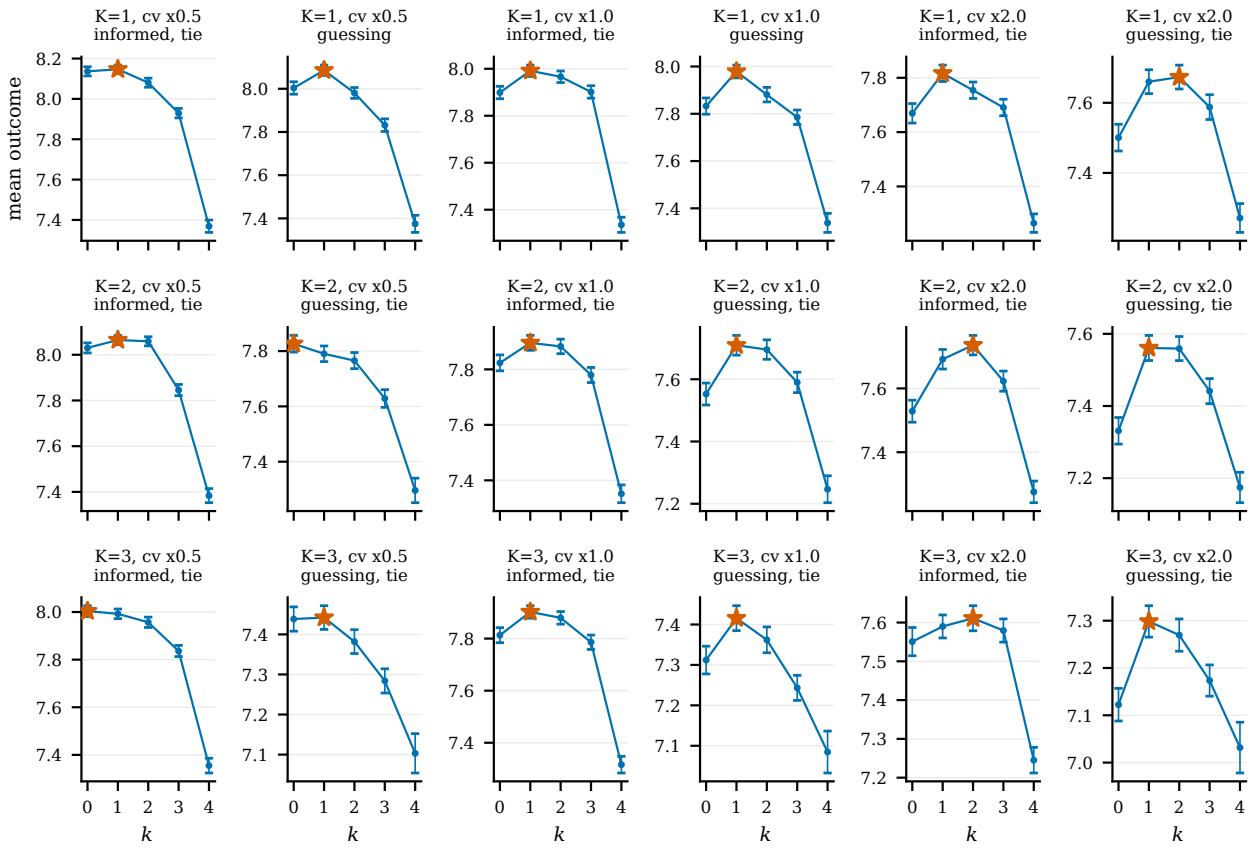


Figure 12: The learnability simulation: mean final pick against k, the slots per round spent re-testing the current best, one panel per setting of ruggedness K, noise and rule, over 200 generated landscapes per setting, with standard errors. The star marks the best k. A panel marked *tie* is one where the paired gap to the runner-up is inside two of its own standard errors.

where $k^* = 1$ and between 0.819 and 0.925 in the other three. Figure 12 is drawn at 200 landscapes, the pre-registered primary, since only that run recorded a standard error for each k.

L Training a policy in Wellocate

Trained on Wellocate's reward alone, a two-parameter rule already settles on a setting that is best or tied for best on generated landscapes (Appendix K). This appendix trains a full policy.

Design. We trained a small policy with proximal policy optimisation (PPO) through Wellocate's released interface. It sees what a language model agent sees: the readings, the outcomes and the remaining budget, and never a true value or the set of viable candidates. At each step it scores a set of candidates. The set holds re-tests of what it has ordered, every single-letter change to the start and to its eight best candidates, and one random untried candidate. A second network picks the nominee and a third estimates value. Its inputs are scaled to the campaign's own readings, so the same networks run on any landscape. Run A trains on generated grid landscapes only. Run B also trains on scrambled GB1 and is tested on PhoQ. Each run has three seeds and keeps the checkpoint with the best score on its validation split. The design, the test and the success criteria were registered before training. The policy had to beat both its own untrained initialisation and

random ordering, each with a 95% interval above zero and every seed positive. It had to do so on the grid test split to show the task is learnable, and on the scrambled GB1 and PhoQ test splits to show transfer.

The registered result. Run A meets both criteria (Table 16). Run B beats its initialisation, and its interval against random ordering includes zero, so it fails the transfer criterion. A rerun decided after the test, with an exploration bonus five times the registered one, meets it on PhoQ: it beats random ordering by 0.50 [0.43, 0.56] with every seed positive (Table 19). The registered intervals resample episodes only. Table 16 gives every interval from a two-level bootstrap that resamples landscapes and then episodes; this check is post-hoc. Run A's criteria all hold under it, and also under an interval across the three seeds. Under the two-level bootstrap, two comparisons with the guessing rule include zero, on the grid and on PhoQ, so we make no claim that the policy beats the guessing rule. Five more seeds of Run A, trained and tested the same way, replicate this: every seed beats its initialisation and random ordering on all three test splits (Table 19).

Robustness. Everything from here on was decided after the registered test. Run A plays every condition without retraining, with the networks of the registered test. Its lead over random ordering holds when the noise is halved or doubled (Table 17). A per-round batch effect, a noise floor fifty times the smallest spread of Appendix A and failed wells leave the scripted policies' best number of re-tests unchanged. Under each of them, three or four re-tests a round stay below that best (Table 21). Spikes on the grid are the one exception. When 2% of readings are three times their value, the informed rule with three re-tests a round becomes the best scripted rule. Its pick minus that of its best setting with fewer re-tests is $+0.03$ $[-0.01, +0.07]$. Run A does not adapt to the spikes. It keeps re-testing about 9.3% of its slots, and its lead over random ordering falls to 0.09 [0.05, 0.13] (Table 17). On GB1 and PhoQ under their real names it behaves as on the scrambled test splits (Table 17). Its lead shrinks on rougher grids and holds at every budget we tried (Table 17). At 4 rounds of 24 it is level with the best scripted rule, with two of its inputs beyond the range it saw in training.

The language models' own campaigns. We replayed Run A on each model's own campaigns: the same landscape, campaign seeds, noise and budget (Table 20). We first checked that each of these landscapes is outside the training and validation splits, by name, by built content and, for the grid, by its sorted fitness values. On the grid Run A beats every model. On scrambled GB1 it is below GPT-6.1 Sol, and on GB1 under real names it is below Opus 4.7.

What the policies learned. We replayed every test episode and read each order back from the environment's record (Table 18). Run A builds from its leaders and rarely re-tests them. On the grid, 61% of its new candidates are one letter from one of its three highest readings. Re-tests of those three take 0.3% of its orders. Most of its re-tests retry an order that failed, 62%, or repeat a candidate first read earlier the same round, 15%. The seeds differ: the third seed re-tests candidates that already have a reading more often. The registered Run B re-tests from the first round, and its share climbs to about four slots in five by the last. Most of its re-tests go to candidates it has read. On GB1 Run A's nominee is often not its highest reading, and that costs it much of its gap to the best scripted rule there.

Table 16: The registered test of the trained policy, 1,500 episodes per split, as a mean over three seeds. Run A trained on generated landscapes only; Run B on generated landscapes and scrambled GB1, tested on PhoQ. Differences are in final pick, with 95% intervals from a two-level bootstrap that resamples landscapes and then episodes (post-hoc; the registered intervals resample episodes only).

	Run A, grid	Run A, GB1	Run A, PhoQ	Run B, PhoQ
trained pick	7.91	4.79	2.15	1.53
re-testing share (%)	9.3	1.8	2.3	68.3
minus initialisation	+0.86 [0.83, 0.89]	+3.32 [3.24, 3.40]	+1.49 [1.44, 1.54]	+0.87 [0.84, 0.91]
minus random	+0.40 [0.36, 0.44]	+1.43 [1.34, 1.53]	+0.65 [0.58, 0.73]	+0.03 [−0.03, 0.09]
minus guessing rule	−0.03 [−0.06, 0.01]	−1.32 [−1.45, −1.20]	+0.07 [−0.01, 0.15]	−0.55 [−0.62, −0.48]
minus informed rule	−0.05 [−0.10, 0.00]	−1.16 [−1.26, −1.07]	−0.52 [−0.63, −0.41]	−1.14 [−1.25, −1.03]
minus model-guided	−0.06 [−0.11, −0.01]	−0.56 [−0.67, −0.45]	+0.27 [0.21, 0.33]	−0.35 [−0.39, −0.31]

Table 17: The trained policy (Run A, without retraining) under every change after the registered test, 1,500 episodes per row, at assay noise unless the row changes it. Rows that would repeat Table 16 are left out. The last two columns are Run A's pick minus random ordering and minus the best scripted rule at its best number of re-tests, on the same episodes. Stress: each round's readings share one multiplicative factor with this standard deviation (batch), an additive noise floor of 2% of the landscape maximum (floor), or 2% of readings replaced by a near-zero value (failed wells) or by three times the reading (spikes). Real names: GB1 and PhoQ under their amino-acid names. At 4 rounds of 24 the scripted rules' re-tests scale with the slots per round, and two of Run A's input features exceed the range seen in training. Intervals as in Table 16; under real names, one landscape each, they resample episodes only.

change	condition	pick	share (%)	minus random	minus best scripted rule
noise, grid	noise halved (3.65%)	8.11	9.2	+0.45 [0.40, 0.49]	−0.04 [−0.08, 0.00]
	noise doubled (14.6%)	7.66	9.0	+0.35 [0.31, 0.40]	−0.09 [−0.14, −0.05]
stress, grid	batch 5%	7.84	9.3	+0.41 [0.36, 0.46]	−0.13 [−0.18, −0.07]
	batch 10%	7.73	9.2	+0.40 [0.35, 0.45]	−0.13 [−0.19, −0.08]
	floor	7.89	9.3	+0.40 [0.35, 0.44]	−0.11 [−0.16, −0.06]
	failed wells	7.90	9.5	+0.39 [0.35, 0.43]	−0.10 [−0.14, −0.06]
	spikes	7.46	9.1	+0.09 [0.05, 0.13]	−0.21 [−0.24, −0.17]
stress, GB1	batch 5%	4.78	1.9	+1.43 [1.33, 1.53]	−1.23 [−1.35, −1.11]
	batch 10%	4.73	1.8	+1.40 [1.28, 1.51]	−1.22 [−1.35, −1.09]
	floor	4.82	1.9	+1.47 [1.37, 1.58]	−1.26 [−1.38, −1.14]
	failed wells	4.75	1.8	+1.42 [1.32, 1.53]	−1.30 [−1.44, −1.16]
	spikes	4.44	1.8	+1.17 [1.06, 1.27]	−1.05 [−1.19, −0.92]
real names	GB1	4.72	1.9	+1.31 [1.24, 1.39]	−1.37 [−1.46, −1.29]
	PhoQ	2.12	2.3	+0.58 [0.53, 0.63]	−0.47 [−0.54, −0.39]
held-out grids	$K = 2$	7.70	8.7	+0.18 [0.12, 0.23]	−0.27 [−0.31, −0.23]
	$K = 3$	7.55	8.3	+0.04 [−0.03, 0.11]	−0.31 [−0.37, −0.25]
budgets, grid	10 rounds of 4	7.83	9.1	+0.42 [0.36, 0.47]	−0.06 [−0.12, −0.01]
	20 rounds of 4	7.97	9.5	+0.40 [0.36, 0.44]	−0.11 [−0.15, −0.07]
	4 rounds of 24	8.01	11.6	+0.37 [0.32, 0.43]	−0.04 [−0.09, 0.00]

Table 18: What the trained policies do on the test splits, as a mean over three seeds. Share: re-tests as a share of the budget. The next three columns split the re-tests: retries of an order that failed, repeats of a candidate first read earlier the same round, and re-tests of one of the three candidates with the highest mean reading. Parent: new candidates one letter from one of those three. Nominee: how often it is the registered selection rule's nominee, and its pick minus that nominee's.

run	split	share (%)	retries (%)	same round (%)	top three (%)	parent (%)	nominee (%)	pick cost
Run A	grid	9.3	62	15	5	61	97	+0.00 [0.00, 0.00]
Run A	GB1	1.8	58	29	4	63	51	−0.71 [−0.75, −0.67]
Run A	PhoQ	2.3	61	23	2	64	89	−0.05 [−0.06, −0.04]
Run B	grid	72.0	3	5	58	43	71	−0.03 [−0.04, −0.02]
Run B	GB1	69.5	3	5	42	35	93	−0.06 [−0.07, −0.05]
Run B	PhoQ	68.3	3	3	39	31	93	−0.02 [−0.02, −0.01]

Table 19: Five more seeds of Run A (three to seven), trained and tested as registered, and Run B retrained with an entropy bonus five times the registered one, tested with the registered code on all three test splits. Run B trains on the grid and GB1, so only PhoQ tests its transfer. Differences in final pick against random ordering, with intervals as in Table 16. Entropies are at the end of training: means over the last updates, averaged over seeds.

	Run A, seeds 3 to 7	Run B, registered	Run B, larger bonus
grid: minus random	+0.42 [0.37, 0.46]	–	+0.32 [0.28, 0.37]
grid: re-testing share (%)	10.3	–	3.2
grid: seeds above random	5 of 5	–	3 of 3
GB1: minus random	+1.63 [1.53, 1.72]	–	+1.66 [1.56, 1.76]
GB1: re-testing share (%)	1.7	–	2.7
GB1: seeds above random	5 of 5	–	3 of 3
PhoQ: minus random	+0.66 [0.59, 0.72]	+0.03 [−0.03, 0.09]	+0.50 [0.43, 0.56]
PhoQ: re-testing share (%)	1.9	68.3	3.1
PhoQ: seeds above random	5 of 5	2 of 3	3 of 3
order-head entropy, nats	5.31	1.06	5.88
nominate-head entropy, nats	0.04	0.03	0.34

Table 20: The trained policy (Run A, without retraining) on each language model's own campaigns, the same landscape, campaign seeds, noise and budget. Pick, with re-testing share; the last column is Run A's pick minus the model's, paired by campaign, with a 95% bootstrap interval over campaigns. Each of these landscapes is outside the training and validation splits.

landscape	model	campaigns	model	Run A	Run A minus model
grid	Opus 4.7	60	7.62 (75%)	7.87 (10%)	+0.25 [0.14, 0.35]
grid	Gemini 2.5 Flash	120	7.70 (94%)	7.86 (9%)	+0.16 [0.11, 0.21]
grid	DeepSeek V3.1	120	7.76 (91%)	7.86 (9%)	+0.10 [0.04, 0.15]
grid	GPT-6.1 Sol	60	7.67 (44%)	7.87 (10%)	+0.19 [0.07, 0.31]
grid	Gemini 3.1 Pro	60	7.69 (14%)	7.87 (10%)	+0.18 [0.07, 0.29]
GB1, scrambled	Opus 4.7	60	4.78 (63%)	4.79 (2%)	+0.02 [−0.37, 0.42]
GB1, scrambled	Gemini 2.5 Flash	60	1.12 (92%)	4.79 (2%)	+3.68 [3.42, 3.92]
GB1, scrambled	DeepSeek V3.1	60	0.84 (96%)	4.79 (2%)	+3.95 [3.67, 4.22]
GB1, scrambled	GPT-6.1 Sol	30	6.36 (19%)	4.67 (2%)	−1.69 [−2.09, −1.35]
GB1, scrambled	Gemini 3.1 Pro	30	4.90 (11%)	4.67 (2%)	−0.24 [−0.82, 0.37]
GB1, real names	Opus 4.7	60	7.59 (51%)	4.75 (2%)	−2.84 [−3.21, −2.46]
GB1, real names	Gemini 2.5 Flash	60	2.41 (45%)	4.75 (2%)	+2.34 [1.93, 2.74]
GB1, real names	DeepSeek V3.1	60	1.27 (92%)	4.75 (2%)	+3.48 [3.11, 3.84]
PhoQ, scrambled	Opus 4.7	30	1.89 (63%)	2.20 (2%)	+0.31 [0.06, 0.58]
PhoQ, scrambled	Gemini 2.5 Flash	30	2.13 (64%)	2.20 (2%)	+0.07 [−0.16, 0.33]
PhoQ, scrambled	DeepSeek V3.1	30	0.83 (90%)	2.20 (2%)	+1.37 [1.17, 1.60]
PhoQ, scrambled	GPT-6.1 Sol	30	2.37 (24%)	2.20 (2%)	−0.17 [−0.44, 0.12]
PhoQ, scrambled	Gemini 3.1 Pro	30	1.72 (15%)	2.20 (2%)	+0.48 [0.24, 0.72]

Table 21: The guessing and informed rules under the stress conditions of Table 17 that keep their best number of re-tests, 1,500 episodes per landscape and condition, at assay noise. Without stress their best is 1 re-test a round on the grid and 0 on GB1. Best k is the best number of re-tests a round, with its re-testing share; three minus best is the pick of three re-tests a round minus that of the best, with intervals as in Table 16. Four re-tests a round are lower still in every row.

landscape	condition	guessing, best k	guessing, three minus best	informed, best k	informed, three minus best
grid	batch 5%	1 (24%)	−0.19 [−0.22, −0.15]	1 (23%)	−0.17 [−0.21, −0.13]
grid	batch 10%	1 (24%)	−0.15 [−0.19, −0.11]	1 (23%)	−0.14 [−0.18, −0.11]
grid	floor	1 (24%)	−0.17 [−0.21, −0.14]	1 (23%)	−0.17 [−0.20, −0.14]
grid	failed wells	1 (24%)	−0.22 [−0.26, −0.17]	1 (23%)	−0.21 [−0.25, −0.17]
GB1	batch 5%	0 (1%)	−2.21 [−2.34, −2.08]	0 (0%)	−1.58 [−1.70, −1.47]
GB1	batch 10%	0 (1%)	−2.16 [−2.30, −2.03]	0 (0%)	−1.48 [−1.59, −1.36]
GB1	floor	0 (1%)	−2.27 [−2.41, −2.14]	0 (0%)	−1.68 [−1.81, −1.55]
GB1	failed wells	0 (1%)	−2.32 [−2.46, −2.16]	0 (0%)	−1.71 [−1.82, −1.59]